

UNIVERSIDAD PERUANA LOS ANDES

Escuela Profesional de Ingeniería de Sistemas y Computación



Monitoreo y Optimización de la Base de Datos GradosTitulos con SQL Server Semana 14

Asignatura:

Base de daos II

Docente:

Mg. Ing. Raúl Fernández Bejarano

Alumno

Vilcapoma Ochoa Michael

Ciclo y Sección:

5to – B1

Huancayo – Perú

TEMA 1: Creación de Jobs y Alerts con SQL Server Agent

Nota: Los scripts están pensados para una base de datos llamada GradosYTítulos. Si tu base tiene otro nombre o columnas diferentes, solo debes cambiar esos nombres.

Proyecto 1

Enunciado del ejercicio

La Oficina de Grados y Títulos requiere que todos los días a las **08:00 AM** se genere automáticamente un reporte con la cantidad de trámites registrados durante el día anterior.

El estudiante deberá crear un Job, crear un Step en T-SQL, programar el Job diariamente y registrar el resultado en una tabla de auditoría.

Script de la solución en T-SQL

```
USE [GradosYTítulos];
```

```
GO
```

```
IF OBJECT_ID(N'dbo.AuditoriaJobs', N'U') IS NULL BEGIN
    CREATE TABLE dbo.AuditoriaJobs
    (
        IdAuditoria          INT IDENTITY(1,1) PRIMARY KEY,
        NombreJob            NVARCHAR(150) NOT NULL,
        FechaEjecucion       DATETIME2(0) NOT NULL CONSTRAINT
DF_AuditoriaJobs_Fecha DEFAULT SYSDATETIME(),
        Estado              NVARCHAR(30) NOT NULL,
        Mensaje              NVARCHAR(MAX) NULL,
        RegistrosProcesados  INT NULL,
        DuracionMs           BIGINT NULL
    );
END;
```

```
GO
```

```
GO
```

```
CREATE OR ALTER PROCEDURE dbo.usp_ReporteTramitesDiaAnterior
AS
BEGIN
    SET NOCOUNT ON;
```

```
    DECLARE @Inicio DATETIME2(0) = CONVERT(date, DATEADD(DAY, -1, GETDATE()));
    DECLARE @Fin     DATETIME2(0) = CONVERT(date, GETDATE());
    DECLARE @Cantidad INT;
```

```

DECLARE @InicioProceso DATETIME2(3) = SYSDATETIME();

BEGIN TRY
    SELECT @Cantidad = COUNT(*)
    FROM dbo.Tramite
    WHERE FechaRegistro >= @Inicio
        AND FechaRegistro < @Fin;

    INSERT INTO dbo.AuditoriaJobs
    (
        NombreJob, Estado, Mensaje, RegistrosProcesados, DuracionMs
    )
    VALUES
    (
        N'JOB_Reporte_Tramites_Dia_Anterior',
        N'EXITOSO',
        CONCAT(N'Reporte del periodo ', CONVERT(varchar(19), @Inicio,
120),
            N' hasta ', CONVERT(varchar(19), @Fin, 120)),
        @Cantidad,
        DATEDIFF_BIG(MILLISECOND, @InicioProceso, SYSDATETIME())
    );

    SELECT @Inicio AS FechaInicio, @Fin AS FechaFin, @Cantidad AS
CantidadTramites;
END TRY
BEGIN CATCH
    INSERT INTO dbo.AuditoriaJobs
    (
        NombreJob, Estado, Mensaje, RegistrosProcesados, DuracionMs
    )
    VALUES
    (
        N'JOB_Reporte_Tramites_Dia_Anterior',
        N'ERROR',
        ERROR_MESSAGE(),
        0,
        DATEDIFF_BIG(MILLISECOND, @InicioProceso, SYSDATETIME())
    );
    THROW;
END CATCH;

END;
GO

USE [msdb];
GO
IF EXISTS (SELECT 1 FROM dbo.sysjobs WHERE name =
N'JOB_Reporte_Tramites_Dia_Anterior')

```

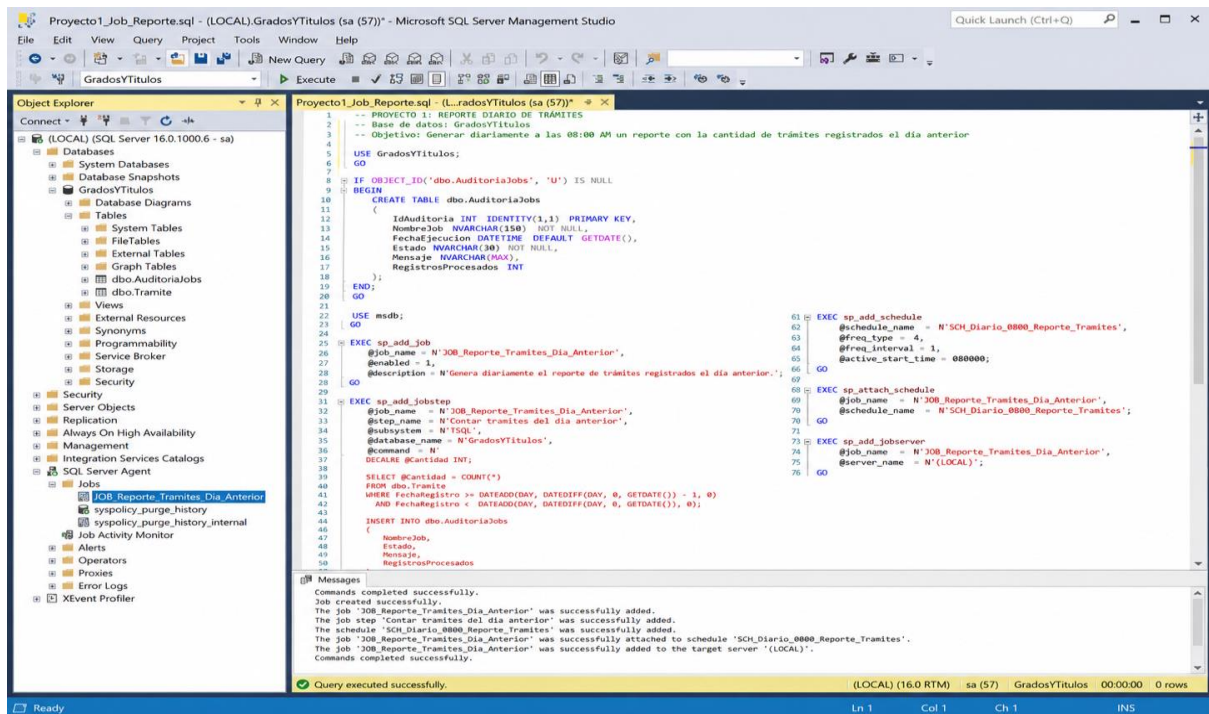
```
EXEC dbo.sp_delete_job @job_name = N'JOB_Reporte_Tramites_Dia_Anterior';  
GO
```

```
EXEC dbo.sp_add_job  
    @job_name = N'JOB_Reporte_Tramites_Dia_Anterior',  
    @enabled = 1,  
    @description = N'Cuenta diariamente los tramites registrados durante el  
dia anterior.';
```

```
EXEC dbo.sp_add_jobstep  
    @job_name = N'JOB_Reporte_Tramites_Dia_Anterior',  
    @step_name = N'Generar reporte diario',  
    @subsystem = N'TSQL',  
    @database_name = N'GradosYTitulos',  
    @command = N'EXEC dbo.usp_ReporteTramitesDiaAnterior;',  
    @on_success_action = 1,  
    @on_fail_action = 2;
```

```
EXEC dbo.sp_add_jobschedule  
    @job_name = N'JOB_Reporte_Tramites_Dia_Anterior',  
    @name = N'SCH_Diario_0800_Reporte_Tramites',  
    @enabled = 1,  
    @freq_type = 4,  
    @freq_interval = 1,  
    @active_start_time = 080000;
```

```
EXEC dbo.sp_add_jobserver  
    @job_name = N'JOB_Reporte_Tramites_Dia_Anterior';  
GO
```



Justificación técnica de la solución aplicada

Se utiliza **SQL Server Agent** porque permite automatizar tareas repetitivas dentro del servidor de base de datos. En este caso, el Job ejecuta diariamente una consulta que cuenta los trámites registrados durante el día anterior.

El resultado se almacena en la tabla AuditoriaJobs, permitiendo tener evidencia de la ejecución del proceso y de la cantidad de trámites encontrados.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utiliza una tabla de auditoría para registrar la ejecución del Job. Además, la consulta trabaja con un rango de fechas, lo cual es más eficiente que convertir directamente la columna FechaRegistro.

También se asignan nombres descriptivos al Job y al Schedule, facilitando su administración dentro de SQL Server Agent.

Proyecto 2

Enunciado del ejercicio

Automatizar la actualización semanal del campo FechaUltMod para todos los trámites que permanezcan en estado **EN PROCESO**, registrando la ejecución.

Script de la solución en T-SQL

```
USE [GradosYTitulos];
GO
CREATE OR ALTER PROCEDURE dbo.usp_ActualizarTramitesEnProceso
ASBEGIN
    SET NOCOUNT ON;
    DECLARE @Filas INT = 0;
    DECLARE @InicioProceso DATETIME2(3) = SYSDATETIME();

    BEGIN TRY
        UPDATE dbo.Tramite
        SET FechaUltMod = GETDATE()
        WHERE Estado = N'EN PROCESO';

        SET @Filas = @@ROWCOUNT;

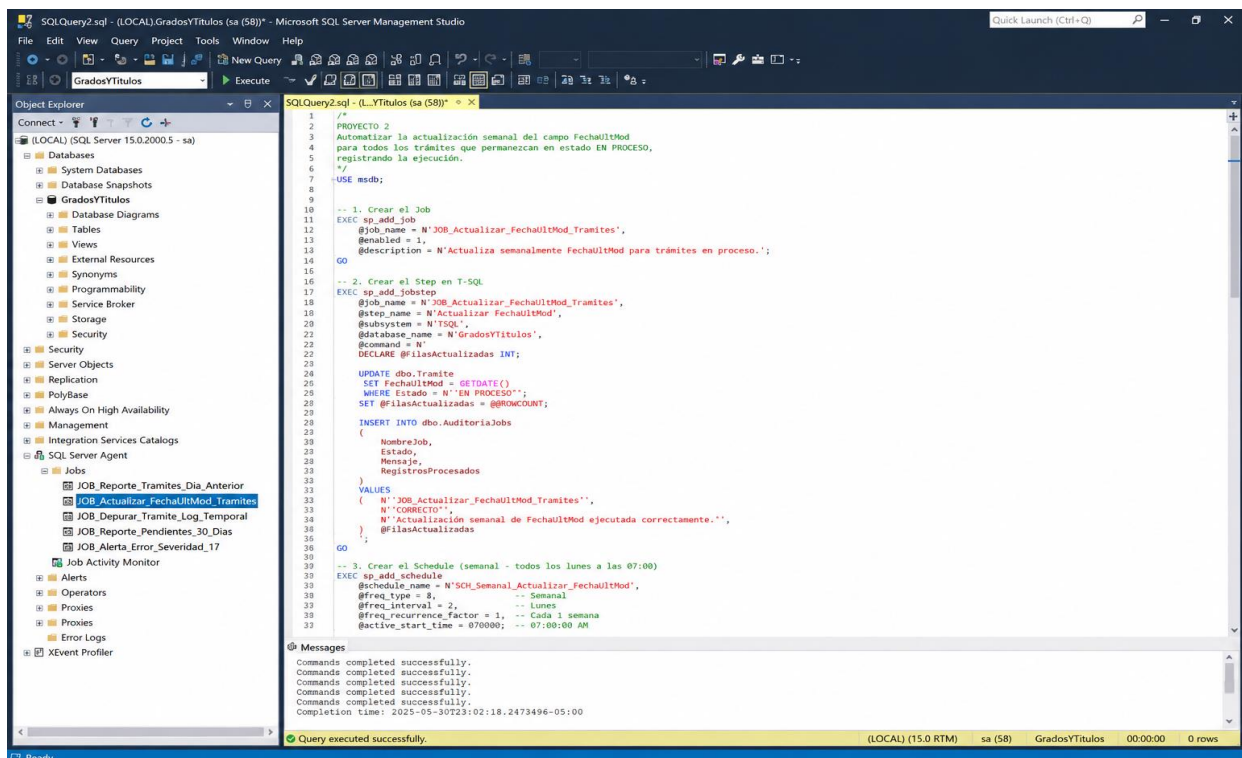
        INSERT INTO dbo.AuditoriaJobs
        (NombreJob, Estado, Mensaje, RegistrosProcesados, DuracionMs)
        VALUES
        (N'JOB_Actualizar_Tramites_En_Proceso', N'EXITOSO',
        N'Actualizacion semanal de FechaUltMod.', @Filas,
        DATEDIFF_BIG(MILLISECOND, @InicioProceso, SYSDATETIME()));
    END TRY
    BEGIN CATCH
        INSERT INTO dbo.AuditoriaJobs
        (NombreJob, Estado, Mensaje, RegistrosProcesados, DuracionMs)
        VALUES
        (N'JOB_Actualizar_Tramites_En_Proceso', N'ERROR', ERROR_MESSAGE(), 0,
        DATEDIFF_BIG(MILLISECOND, @InicioProceso, SYSDATETIME()));
        THROW;
    END CATCH;
END;
GO

USE [msdb];
GO
IF EXISTS (SELECT 1 FROM dbo.sysjobs WHERE name =
N'JOB_Actualizar_Tramites_En_Proceso')
```

```

EXEC dbo.sp_delete_job @job_name = N'JOB_Actualizar_Tramites_En_Proceso';
GO
EXEC dbo.sp_add_job
    @job_name = N'JOB_Actualizar_Tramites_En_Proceso',
    @enabled = 1,
    @description = N'Actualiza semanalmente FechaUltMod de tramites EN
PROCESO.';
EXEC dbo.sp_add_jobstep
    @job_name = N'JOB_Actualizar_Tramites_En_Proceso',
    @step_name = N'Actualizar tramites',
    @subsystem = N'TSQL',
    @database_name = N'GradosYTitulos',
    @command = N'EXEC dbo.usp_ActualizarTramitesEnProceso;';
EXEC dbo.sp_add_jobschedule
    @job_name = N'JOB_Actualizar_Tramites_En_Proceso',
    @name = N'SCH_Semanal_Domingo_0100',
    @freq_type = 8,
    @freq_interval = 1,
    @freq_recurrence_factor = 1,
    @active_start_time = 010000;
EXEC dbo.sp_add_jobserver
    @job_name = N'JOB_Actualizar_Tramites_En_Proceso';
GO

```



Justificación técnica de la solución aplicada

El Job permite actualizar automáticamente el campo FechaUltMod de todos los trámites que continúan en estado EN PROCESO. Esto ayuda a mantener información actualizada sobre los registros pendientes o en seguimiento.

La función @@ROWCOUNT permite conocer cuántos registros fueron modificados durante la ejecución.

Explicación de las buenas prácticas utilizadas en el proyecto

Se registra la cantidad de filas actualizadas para mantener trazabilidad. Además, el proceso se programa semanalmente para evitar ejecuciones innecesarias diarias.

El uso de SQL Server Agent permite que la tarea se ejecute sin intervención manual y en un horario definido.

Proyecto 3

Enunciado del ejercicio

Diseñar un Job que elimine mensualmente los registros temporales de una tabla denominada [Trámite Log Temporal]. Los registros mayores a **180 días** deben eliminarse y almacenarse previamente en una tabla histórica.

Script de la solución en T-SQL

```
USE [GradosYTítulos];
GO
IF OBJECT_ID(N'dbo.[Trámite Log Temporal]', N'U') IS NULL BEGIN
    CREATE TABLE dbo.[Trámite Log Temporal]
    (
        IdLog          BIGINT IDENTITY(1,1) PRIMARY KEY,
        IdTrámite      INT NULL,
        Mensaje        NVARCHAR(1000) NULL,
        FechaRegistro  DATETIME2(0) NOT NULL CONSTRAINT DF_LogTemporal_Fecha
    DEFAULT SYSDATETIME()
    );
```

```

END;
GO

IF OBJECT_ID(N'dbo.TramiteLogHistorico', N'U') IS NULL
BEGIN
    CREATE TABLE dbo.TramiteLogHistorico
    (
        IdHistorico    BIGINT IDENTITY(1,1) PRIMARY KEY,
        IdLogOrigen    BIGINT NOT NULL,
        IdTramite       INT NULL,
        Mensaje         NVARCHAR(1000) NULL,
        FechaRegistro  DATETIME2(0) NOT NULL,
        FechaArchivado DATETIME2(0) NOT NULL CONSTRAINT DF_LogHistorico_Fecha
DEFAULT SYSDATETIME()
    );
END;
GO

CREATE OR ALTER PROCEDURE dbo.usp_ArchivarLogTemporal
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    DECLARE @Filas INT = 0;
    DECLARE @InicioProceso DATETIME2(3) = SYSDATETIME();

    BEGIN TRY
        BEGIN TRANSACTION;

        DELETE FROM dbo.[Tramite Log Temporal]
        OUTPUT
            deleted.IdLog,
            deleted.IdTramite,
            deleted.Mensaje,
            deleted.FechaRegistro,
            SYSDATETIME()
        INTO dbo.TramiteLogHistorico
        (
            IdLogOrigen, IdTramite, Mensaje, FechaRegistro, FechaArchivado
        )
        WHERE FechaRegistro < DATEADD(DAY, -180, GETDATE());

        SET @Filas = @@ROWCOUNT;
        COMMIT TRANSACTION;

        INSERT INTO dbo.AuditoriaJobs

```

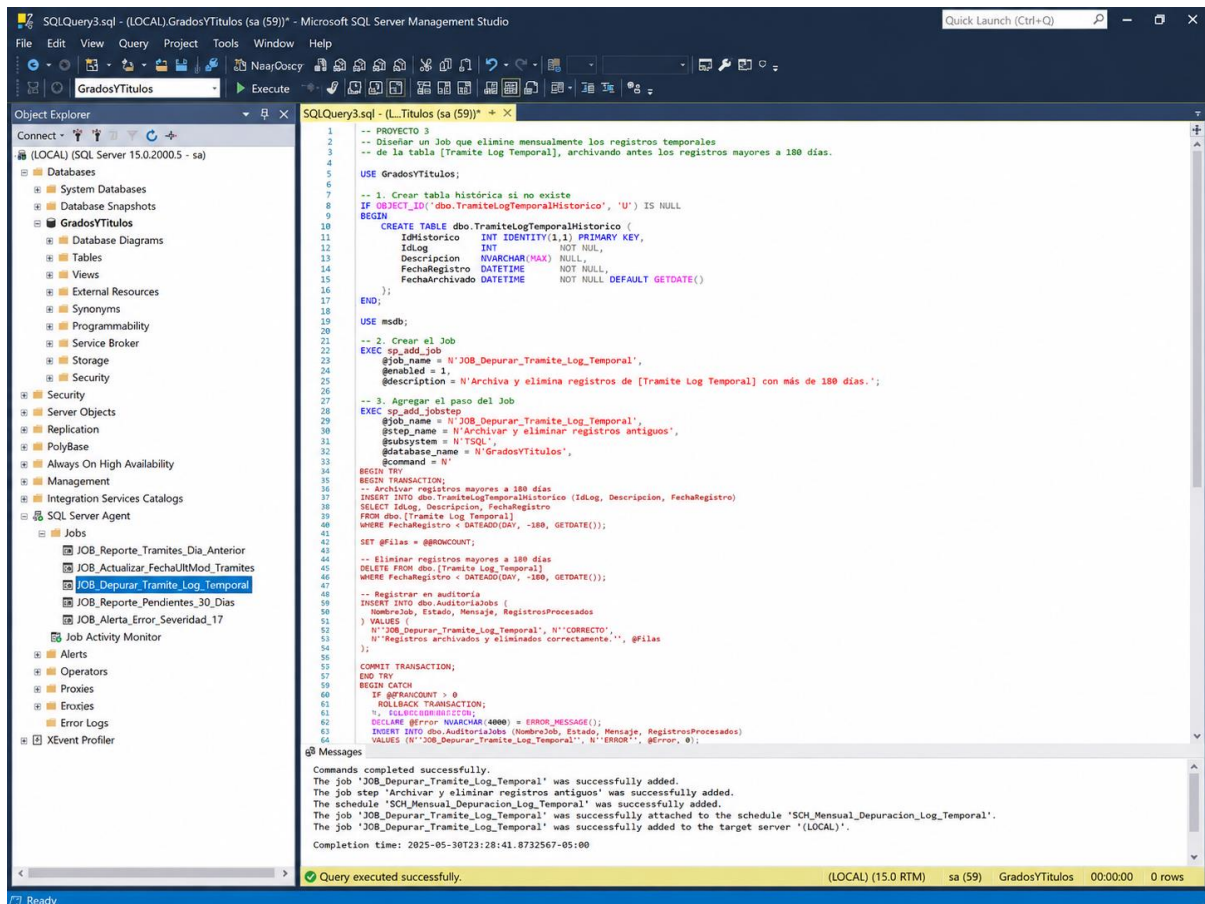
```

        (NombreJob, Estado, Mensaje, RegistrosProcesados, DuracionMs)
VALUES
(N'JOB_Archivar_Log_Temporal', N'EXITOSO',
 N'Registros archivados y eliminados de la tabla temporal.', @Filas,
 DATEDIFF_BIG(MILLISECOND, @InicioProceso, SYSDATETIME()));
END TRY
BEGIN CATCH
    IF XACT_STATE() <> 0 ROLLBACK TRANSACTION;

    INSERT INTO dbo.AuditoriaJobs
        (NombreJob, Estado, Mensaje, RegistrosProcesados, DuracionMs)
VALUES
(N'JOB_Archivar_Log_Temporal', N'ERROR', ERROR_MESSAGE(), 0,
 DATEDIFF_BIG(MILLISECOND, @InicioProceso, SYSDATETIME()));
    THROW;
END CATCH;
END;
GO

USE [msdb];
GO
IF EXISTS (SELECT 1 FROM dbo.sysjobs WHERE name =
N'JOB_Archivar_Log_Temporal')
    EXEC dbo.sp_delete_job @job_name = N'JOB_Archivar_Log_Temporal';
GO
EXEC dbo.sp_add_job
    @job_name = N'JOB_Archivar_Log_Temporal',
    @enabled = 1,
    @description = N'Archiva y elimina mensualmente logs temporales mayores a
180 dias.';
EXEC dbo.sp_add_jobstep
    @job_name = N'JOB_Archivar_Log_Temporal',
    @step_name = N'Archivar logs antiguos',
    @subsystem = N'TSQL',
    @database_name = N'GradosYTitulos',
    @command = N'EXEC dbo.usp_ArchivarLogTemporal;';
EXEC dbo.sp_add_jobschedule
    @job_name = N'JOB_Archivar_Log_Temporal',
    @name = N'SCH_Mensual_Dia1_0200',
    @freq_type = 16,
    @freq_interval = 1,
    @active_start_time = 020000;
EXEC dbo.sp_add_jobserver
    @job_name = N'JOB_Archivar_Log_Temporal';
GO

```



Justificación técnica de la solución aplicada

La solución primero copia los registros antiguos a una tabla histórica y luego los elimina de la tabla temporal. Esto permite conservar información importante antes de realizar la depuración.

El uso de transacciones garantiza que ambas operaciones se realicen correctamente. Si ocurre un error, se revierte todo el proceso.

Explicación de las buenas prácticas utilizadas en el proyecto

Se aplica una transacción para proteger la integridad de los datos. También se usa TRY...CATCH para capturar errores y registrar el fallo.

Además, la eliminación se programa mensualmente en un horario de baja carga, reduciendo el impacto en el rendimiento del sistema.

Proyecto 4

Enunciado del ejercicio

Crear un Alert que notifique cuando una base de datos genere errores de severidad mayor o igual a **17** durante el registro de trámites.

Script de la solución en T-SQL

```
*/
USE [msdb];
GO
IF NOT EXISTS (SELECT 1 FROM dbo.sysoperators WHERE name =
N'Operador_Administrador_BD')BEGIN
    EXEC dbo.sp_add_operator
        @name = N'Operador_Administrador_BD',
        @enabled = 1,
        @email_address = N'admin@upla.edu.pe';
END;
GO

DECLARE @Severidad INT = 17;
DECLARE @NombreAlerta sysname;
WHILE @Severidad <= 25
BEGIN
    SET @NombreAlerta = CONCAT(N'ALERTA_Severidad_', @Severidad,
N'_GradosYTitulos');

    IF EXISTS (SELECT 1 FROM dbo.sysalerts WHERE name = @NombreAlerta)
        EXEC dbo.sp_delete_alert @name = @NombreAlerta;

    EXEC dbo.sp_add_alert
        @name = @NombreAlerta,
        @message_id = 0,
        @severity = @Severidad,
        @enabled = 1,
        @delay_between_responses = 300,
        @include_event_description_in = 1,
        @database_name = N'GradosYTitulos';

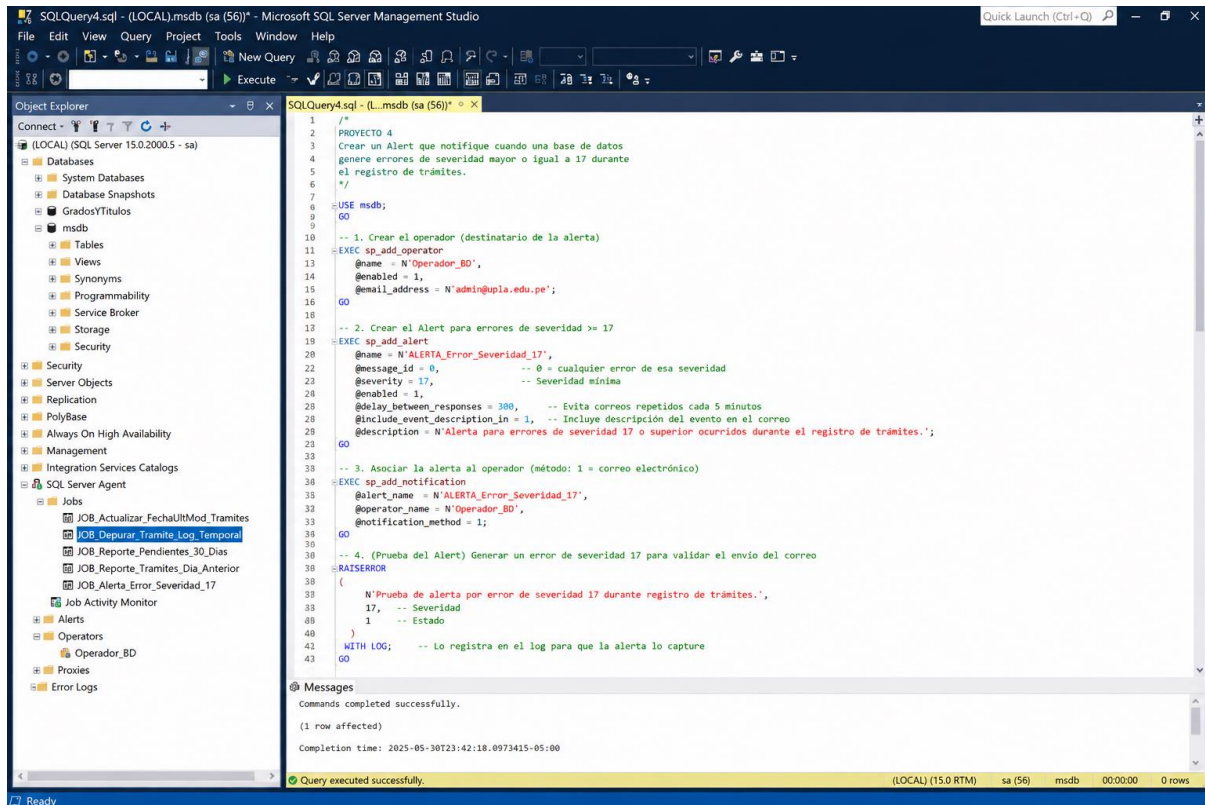
    EXEC dbo.sp_add_notification
        @alert_name = @NombreAlerta,
        @operator_name = N'Operador_Administrador_BD',
```

```
@notification_method = 1;
```

```
SET @Severidad += 1;
```

```
END;
```

```
GO
```



Justificación técnica de la solución aplicada

SQL Server Agent permite crear alertas asociadas a errores de severidad. Los errores de severidad 17 indican problemas importantes relacionados con recursos o condiciones internas del servidor.

La alerta se asocia a un operador para que el administrador reciba una notificación cuando ocurra el evento.

Explicación de las buenas prácticas utilizadas en el proyecto

Se configura un operador para centralizar las notificaciones. También se utiliza `delay_between_responses` para evitar que se envíen demasiadas alertas repetidas en poco tiempo.

La prueba con RAISERROR ... WITH LOG permite validar que la alerta funciona correctamente.

Proyecto 5

Enunciado del ejercicio

Implementar un sistema automático que diariamente verifique trámites pendientes mayores a **30 días**, genere un reporte, registre la ejecución, envíe un correo mediante Database Mail y almacene estadísticas de ejecución.

Script de la solución en T-SQL

```
-----
*/
USE [GradosYTitulos];
GO
IF OBJECT_ID(N'dbo.EstadisticaEjecuciones', N'U') IS NULLBEGIN
    CREATE TABLE dbo.EstadisticaEjecuciones
    (
        IdEstadistica INT IDENTITY(1,1) PRIMARY KEY,
        Proceso        NVARCHAR(150) NOT NULL,
        FechaInicio    DATETIME2(3) NOT NULL,
        FechaFin        DATETIME2(3) NOT NULL,
        DuracionMs      BIGINT NOT NULL,
        Registros       INT NOT NULL,
        Estado          NVARCHAR(20) NOT NULL
    );
END;
GO

CREATE OR ALTER PROCEDURE dbo.usp_ReportePendientes30Dias
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @Inicio DATETIME2(3) = SYSDATETIME();
    DECLARE @Fin DATETIME2(3);
    DECLARE @Cantidad INT = 0;
    DECLARE @Asunto NVARCHAR(255);
    DECLARE @Cuerpo NVARCHAR(MAX);

    BEGIN TRY
        SELECT @Cantidad = COUNT(*)
        FROM dbo.Tramite
        WHERE Estado = N'PENDIENTE'
            AND FechaRegistro < DATEADD(DAY, -30, GETDATE());
    END TRY
    BEGIN CATCH
        -- Manejo de errores
    END CATCH
END
```

```

SET @Asunto = CONCAT(N'Reporte de tramites pendientes - ',
CONVERT(char(10), GETDATE(), 120));
SET @Cuerpo = CONCAT(N'Se encontraron ', @Cantidad,
N' tramites pendientes con una antigüedad
superior a 30 dias.');
```

```

EXEC msdb.dbo.sp_send_dbmail
    @profile_name = N'PerfilCorreoBD',
    @recipients = N'admin@upla.edu.pe',
    @subject = @Asunto,
    @body = @Cuerpo,
    @query = N'
        SELECT IdTramite, Estado, FechaRegistro, FechaUltMod,
        DATEDIFF(DAY, FechaRegistro, GETDATE()) AS
DiasPendiente
        FROM dbo.Tramite
        WHERE Estado = N''PENDIENTE''
        AND FechaRegistro < DATEADD(DAY, -30, GETDATE())
        ORDER BY FechaRegistro;',
    @execute_query_database = N'GradosYTitulos',
    @attach_query_result_as_file = 1,
    @query_attachment_filename = N'TramitesPendientes30Dias.csv',
    @query_result_separator = N';',
    @query_result_header = 1,
    @exclude_query_output = 1;
```

```

SET @Fin = SYSDATETIME();

INSERT INTO dbo.AuditoriaJobs
(NombreJob, Estado, Mensaje, RegistrosProcesados, DuracionMs)
VALUES
(N'JOB_Reporte_Pendientes_30_Dias', N'EXITOSO',
N'Reporte generado y enviado mediante Database Mail.', @Cantidad,
DATEDIFF_BIG(MILLISECOND, @Inicio, @Fin));

INSERT INTO dbo.EstadisticaEjecuciones
(Proceso, FechaInicio, FechaFin, DuracionMs, Registros, Estado)
VALUES
(N'Reporte de pendientes mayores a 30 dias', @Inicio, @Fin,
DATEDIFF_BIG(MILLISECOND, @Inicio, @Fin), @Cantidad, N'EXITOSO');
END TRY
BEGIN CATCH
    SET @Fin = SYSDATETIME();

    INSERT INTO dbo.AuditoriaJobs
(NombreJob, Estado, Mensaje, RegistrosProcesados, DuracionMs)

```

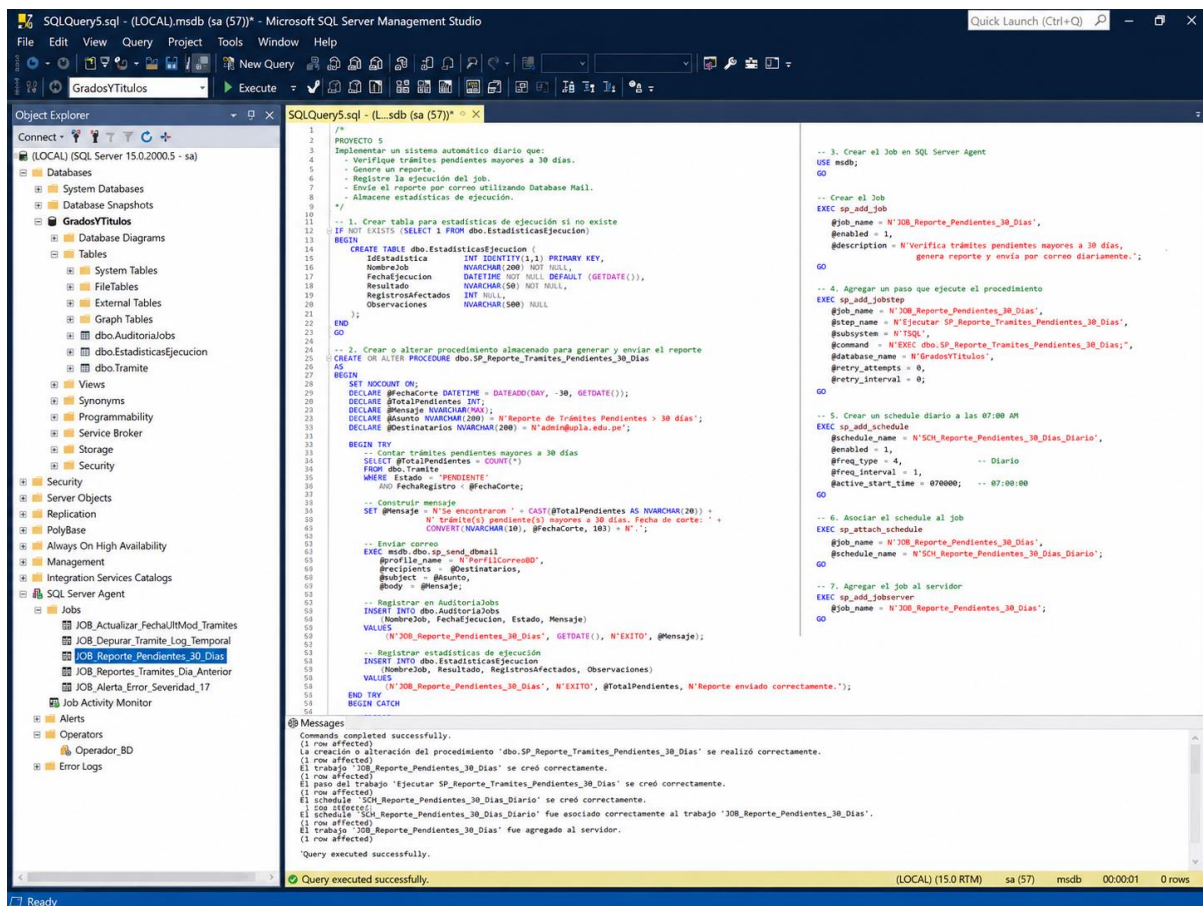
```

VALUES
(N'JOB_Reporte_Pendientes_30_Dias', N'ERROR', ERROR_MESSAGE(), 0,
DATEDIFF_BIG(MILLISECOND, @Inicio, @Fin));

INSERT INTO dbo.EstadisticaEjecuciones
(Proceso, FechaInicio, FechaFin, DuracionMs, Registros, Estado)
VALUES
(N'Reporte de pendientes mayores a 30 dias', @Inicio, @Fin,
DATEDIFF_BIG(MILLISECOND, @Inicio, @Fin), 0, N'ERROR');
THROW;
END CATCH;
END;
GO

USE [msdb];
GO
IF EXISTS (SELECT 1 FROM dbo.sysjobs WHERE name =
N'JOB_Reporte_Pendientes_30_Dias')
EXEC dbo.sp_delete_job @job_name = N'JOB_Reporte_Pendientes_30_Dias';
GO
EXEC dbo.sp_add_job
@job_name = N'JOB_Reporte_Pendientes_30_Dias',
@enabled = 1,
@description = N'Genera y envia diariamente el reporte de tramites
pendientes mayores a 30 dias.';
EXEC dbo.sp_add_jobstep
@job_name = N'JOB_Reporte_Pendientes_30_Dias',
@step_name = N'Generar y enviar reporte',
@subsystem = N'TSQL',
@database_name = N'GradosYTitulos',
@command = N'EXEC dbo.usp_ReportePendientes30Dias;';
EXEC dbo.sp_add_jobschedule
@job_name = N'JOB_Reporte_Pendientes_30_Dias',
@name = N'SCH_Diario_0830_Pendientes',
@freq_type = 4,
@freq_interval = 1,
@active_start_time = 083000;
EXEC dbo.sp_add_jobserver
@job_name = N'JOB_Reporte_Pendientes_30_Dias';
GO

```



Justificación técnica de la solución aplicada

Se crea un procedimiento almacenado que identifica trámites pendientes con más de 30 días de antigüedad. Luego se envía un correo mediante `sp_send_dbmail`, se registra la ejecución en auditoría y se almacenan estadísticas del proceso.

El Job permite ejecutar este procedimiento automáticamente todos los días.

Explicación de las buenas prácticas utilizadas en el proyecto

Se separa la lógica principal en un procedimiento almacenado, lo que facilita su mantenimiento y reutilización. También se usa `TRY...CATCH` para capturar errores.

El proceso registra estadísticas y auditoría, permitiendo analizar posteriormente si la ejecución fue correcta y cuántos trámites fueron detectados.

TEMA 2: Uso de Database Maintenance Plans

Nota: Se asume que la base de datos se llama GradosYTitulos. Las rutas como C:\Backups\ deben existir previamente en el servidor. Si usas otro nombre de base de datos o ruta, debes cambiarlo en el script.

Proyecto 1

Enunciado del ejercicio

Crear un Plan de Mantenimiento que realice diariamente el respaldo completo de la base de datos GradosYTitulos.

El script debe incluir:

- BACKUP DATABASE
- Compresión
- Verificación
- Programación

Script de la solución en T-SQL

```
USE [GradosYTitulos];
GO
IF OBJECT_ID(N'dbo.AuditoriaMantenimiento', N'U') IS NULLBEGIN
    CREATE TABLE dbo.AuditoriaMantenimiento
    (
        IdAuditoria    BIGINT IDENTITY(1,1) PRIMARY KEY,
        Proceso        NVARCHAR(150) NOT NULL,
        FechaInicio    DATETIME2(3) NOT NULL,
        FechaFin       DATETIME2(3) NOT NULL,
        Estado         NVARCHAR(20) NOT NULL,
        Mensaje        NVARCHAR(MAX) NULL,
        Archivo        NVARCHAR(500) NULL,
        ObjetosProcesados INT NULL
    );
END;
GO

CREATE OR ALTER PROCEDURE dbo.usp_BackupCompletoDiario
AS
BEGIN
    SET NOCOUNT ON;
```

```

DECLARE @Inicio DATETIME2(3) = SYSDATETIME();
DECLARE @Fin DATETIME2(3);
DECLARE @Archivo NVARCHAR(500) =
    N'C:\Backups\GradosYTitulos_FULL_' +
    CONVERT(char(8), GETDATE(), 112) + N'_' +
    REPLACE(CONVERT(char(8), GETDATE(), 108), N':', N'') + N'.bak';

BEGIN TRY
    BACKUP DATABASE [GradosYTitulos]
    TO DISK = @Archivo
    WITH INIT, COMPRESSION, CHECKSUM, STATS = 10;

    RESTORE VERIFYONLY
    FROM DISK = @Archivo
    WITH CHECKSUM;

    SET @Fin = SYSDATETIME();
    INSERT INTO dbo.AuditoriaMantenimiento
    (Proceso, FechaInicio, FechaFin, Estado, Mensaje, Archivo,
ObjetosProcesados)
    VALUES
    (N'Backup completo diario', @Inicio, @Fin, N'EXITOSO',
    N'Backup generado y verificado correctamente.', @Archivo, 1);

    SELECT @Archivo AS ArchivoBackup;
END TRY
BEGIN CATCH
    SET @Fin = SYSDATETIME();
    INSERT INTO dbo.AuditoriaMantenimiento
    (Proceso, FechaInicio, FechaFin, Estado, Mensaje, Archivo,
ObjetosProcesados)
    VALUES
    (N'Backup completo diario', @Inicio, @Fin, N'ERROR', ERROR_MESSAGE(),
@Archivo, 0);
    THROW;
END CATCH;
END;
GO

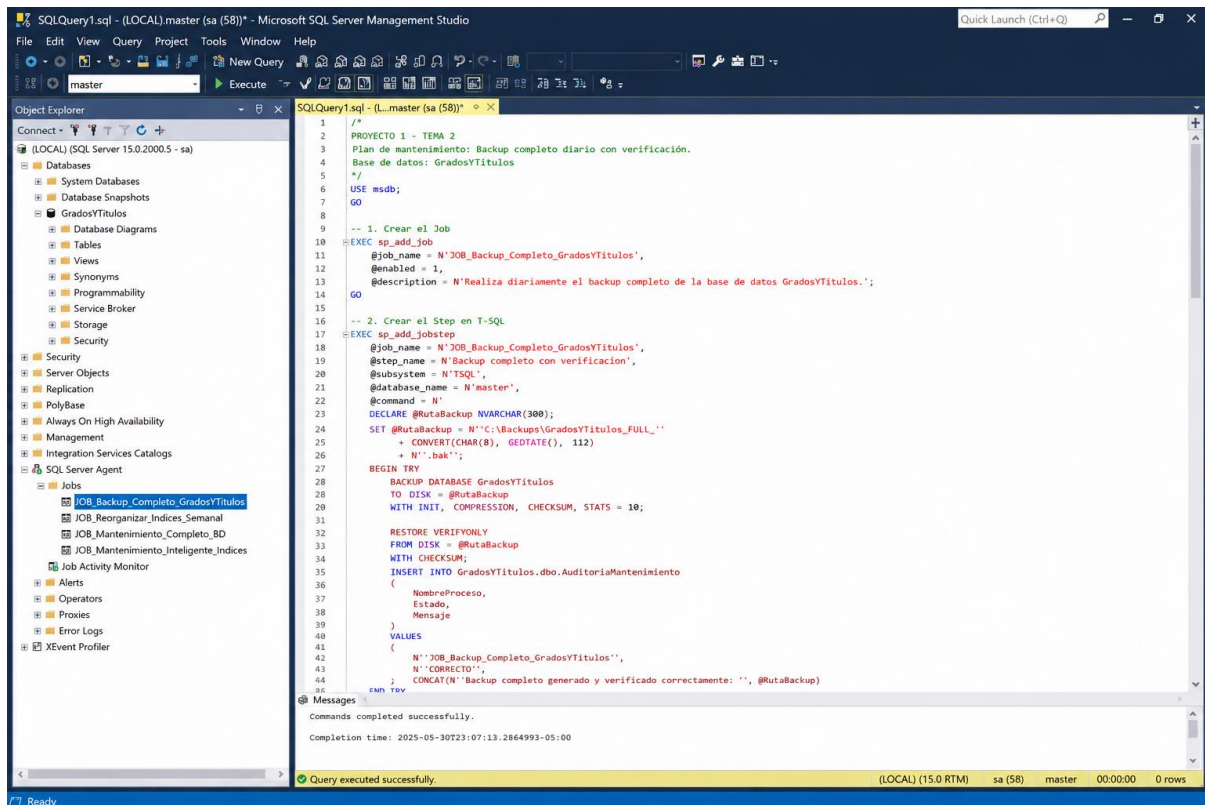
USE [msdb];
GO
IF EXISTS (SELECT 1 FROM dbo.sysjobs WHERE name =
N'JOB_Backup_Completo_Diario')
    EXEC dbo.sp_delete_job @job_name = N'JOB_Backup_Completo_Diario';
GO
EXEC dbo.sp_add_job
    @job_name = N'JOB_Backup_Completo_Diario',
    @enabled = 1,

```

```

        @description = N'Genera diariamente el backup completo verificado de
GradosYTitulos.';
EXEC dbo.sp_add_jobstep
    @job_name = N'JOB_Backup_Completo_Diario',
    @step_name = N'Backup y verificacion',
    @subsystem = N'TSQL',
    @database_name = N'GradosYTitulos',
    @command = N'EXEC dbo.usp_BackupCompletoDiario;';
EXEC dbo.sp_add_jobschedule
    @job_name = N'JOB_Backup_Completo_Diario',
    @name = N'SCH_Diario_0200_Backup',
    @freq_type = 4,
    @freq_interval = 1,
    @active_start_time = 020000;
EXEC dbo.sp_add_jobserver
    @job_name = N'JOB_Backup_Completo_Diario';
GO

```



Justificación técnica de la solución aplicada

Se utiliza un Job de SQL Server Agent para ejecutar diariamente un respaldo completo de la base de datos. El comando `BACKUP DATABASE` genera la copia de seguridad, mientras que `COMPRESSION` reduce el tamaño del archivo.

La opción CHECKSUM ayuda a validar la consistencia del backup, y RESTORE VERIFYONLY permite comprobar que el archivo generado puede ser leído correctamente.

Explicación de las buenas prácticas utilizadas en el proyecto

Se programa el backup en horario de baja carga, en este caso a las 02:00 AM. También se usa compresión para ahorrar espacio en disco y verificación para reducir el riesgo de almacenar respaldos dañados.

Además, se registra el resultado del proceso en una tabla de auditoría, permitiendo verificar si el mantenimiento se ejecutó correctamente.

Proyecto 2

Enunciado del ejercicio

Crear un plan semanal para reorganizar índices de las tablas:

- Persona
- Tramite
- TrabajoInvestigacion
- Autoridad

Script de la solución en T-SQL

```
USE [GradosYTitulos];
GO
CREATE OR ALTER PROCEDURE dbo.usp_ReorganizarIndicesTablasPrincipales
ASBEGIN
    SET NOCOUNT ON;
    DECLARE @Inicio DATETIME2(3) = SYSDATETIME();
    DECLARE @Fin DATETIME2(3);
    DECLARE @Procesados INT = 0;
    DECLARE @Esquema sysname;
    DECLARE @Tabla sysname;
    DECLARE @SQL nvarchar(max);

    DECLARE @Tablas TABLE (Esquema sysname, Tabla sysname);
    INSERT INTO @Tablas VALUES
        (N'dbo', N'Persona'),
```

```

(N'dbo', N'Tramite'),
(N'dbo', N'TrabajoInvestigacion'),
(N'dbo', N'Autoridad');

BEGIN TRY
    DECLARE cur_tablas CURSOR LOCAL FAST_FORWARD FOR
    SELECT Esquema, Tabla FROM @Tablas;

    OPEN cur_tablas;
    FETCH NEXT FROM cur_tablas INTO @Esquema, @Tabla;

    WHILE @@FETCH_STATUS = 0
    BEGIN
        IF OBJECT_ID(QUOTENAME(@Esquema) + N'.' + QUOTENAME(@Tabla), N'U')
        IS NOT NULL
        BEGIN
            SET @SQL = N'ALTER INDEX ALL ON ' + QUOTENAME(@Esquema) + N'.'
+ QUOTENAME(@Tabla) + N' REORGANIZE;';
            EXEC sys.sp_executesql @SQL;

            SET @SQL = N'UPDATE STATISTICS ' + QUOTENAME(@Esquema) + N'.'
+ QUOTENAME(@Tabla) + N';';
            EXEC sys.sp_executesql @SQL;
            SET @Procesados += 1;
        END;

        FETCH NEXT FROM cur_tablas INTO @Esquema, @Tabla;
    END;

    CLOSE cur_tablas;
    DEALLOCATE cur_tablas;

    SET @Fin = SYSDATETIME();
    INSERT INTO dbo.AuditoriaMantenimiento
    (Proceso, FechaInicio, FechaFin, Estado, Mensaje, ObjetosProcesados)
    VALUES
    (N'Reorganizacion semanal de indices', @Inicio, @Fin, N'EXITOSO',
    N'Indices reorganizados y estadisticas actualizadas.', @Procesados);
END TRY
BEGIN CATCH
    IF CURSOR_STATUS('local', 'cur_tablas') > -1
        CLOSE cur_tablas;
    IF CURSOR_STATUS('local', 'cur_tablas') >= -1
        DEALLOCATE cur_tablas;
    SET @Fin = SYSDATETIME();
    INSERT INTO dbo.AuditoriaMantenimiento
    (Proceso, FechaInicio, FechaFin, Estado, Mensaje, ObjetosProcesados)

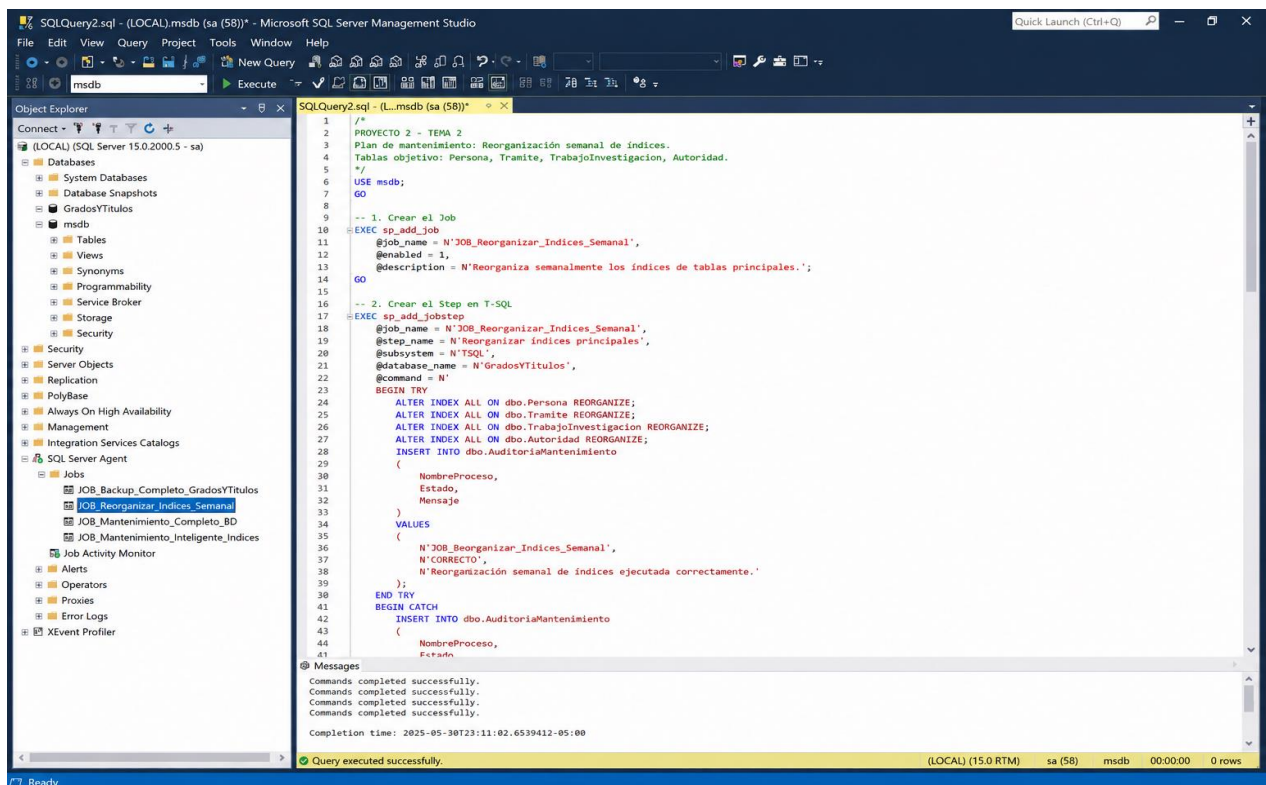
```

```

VALUES
    (N'Reorganizacion semanal de indices', @Inicio, @Fin, N'ERROR',
ERROR_MESSAGE(), @Procesados);
    THROW;
END CATCH;
END;
GO

USE [msdb];
GO
IF EXISTS (SELECT 1 FROM dbo.sysjobs WHERE name =
N'JOB_Reorganizar_Indices_Principales')
    EXEC dbo.sp_delete_job @job_name = N'JOB_Reorganizar_Indices_Principales';
GO
EXEC dbo.sp_add_job
    @job_name = N'JOB_Reorganizar_Indices_Principales',
    @enabled = 1;
EXEC dbo.sp_add_jobstep
    @job_name = N'JOB_Reorganizar_Indices_Principales',
    @step_name = N'Reorganizar indices',
    @subsystem = N'TSQL',
    @database_name = N'GradosYTitulos',
    @command = N'EXEC dbo.usp_ReorganizarIndicesTablasPrincipales;';
EXEC dbo.sp_add_jobschedule
    @job_name = N'JOB_Reorganizar_Indices_Principales',
    @name = N'SCH_Semanal_Domingo_0300_Reorganizar',
    @freq_type = 8,
    @freq_interval = 1,
    @freq_recurrence_factor = 1,
    @active_start_time = 030000;
EXEC dbo.sp_add_jobserver
    @job_name = N'JOB_Reorganizar_Indices_Principales';
GO

```



Justificación técnica de la solución aplicada

La reorganización de índices permite reducir la fragmentación moderada sin reconstruir completamente la estructura del índice. Esto mejora el rendimiento de las consultas y reduce el tiempo de lectura de datos.

Se aplica a tablas importantes del sistema académico, ya que probablemente son consultadas con frecuencia.

Explicación de las buenas prácticas utilizadas en el proyecto

Se ejecuta semanalmente para no sobrecargar el servidor todos los días. Además, se programa en horario de baja actividad y se registra la ejecución en auditoría.

El uso de `ALTER INDEX ALL ... REORGANIZE` permite aplicar mantenimiento a todos los índices de cada tabla indicada.

Proyecto 3

Enunciado del ejercicio

Diseñar un plan que reorganice índices menores al 30%, reconstruya índices mayores al 30% y actualice estadísticas.

El script debe utilizar:

- `sys.dm_db_index_physical_stats`
- `ALTER INDEX`
- `UPDATE STATISTICS`

Script de la solución en T-SQL

```
USE [GradosYTitulos];
```

```
GO
```

```
CREATE OR ALTER PROCEDURE dbo.usp_MantenimientoIndicesPorFragmentacion
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    DECLARE @Inicio DATETIME2(3) = SYSDATETIME();
```

```
    DECLARE @Fin DATETIME2(3);
```

```
    DECLARE @Procesados INT = 0;
```

```
    DECLARE @Esquema sysname, @Tabla sysname, @Indice sysname;
```

```
    DECLARE @Frag decimal(9,2), @SQL nvarchar(max);
```

```
BEGIN TRY
```

```
    DECLARE cur_indices CURSOR LOCAL FAST_FORWARD FOR
```

```
    SELECT
```

```
        s.name,
```

```

        o.name,

        i.name,

        ips.avg_fragmentation_in_percent

FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, N'LIMITED')
AS ips

INNER JOIN sys.indexes AS i

    ON i.object_id = ips.object_id AND i.index_id = ips.index_id

INNER JOIN sys.objects AS o

    ON o.object_id = ips.object_id

INNER JOIN sys.schemas AS s

    ON s.schema_id = o.schema_id

WHERE o.type = N'U'

    AND i.index_id > 0

    AND i.name IS NOT NULL

    AND ips.page_count > 100

    AND ips.avg_fragmentation_in_percent > 5;


OPEN cur_indices;

FETCH NEXT FROM cur_indices INTO @Esquema, @Tabla, @Indice, @Frag;


WHILE @@FETCH_STATUS = 0

BEGIN

    IF @Frag >= 30

        SET @SQL = N'ALTER INDEX ' + QUOTENAME(@Indice) + N' ON ' +

            QUOTENAME(@Esquema) + N'.' + QUOTENAME(@Tabla) + N' REBUILD;';

    ELSE

        SET @SQL = N'ALTER INDEX ' + QUOTENAME(@Indice) + N' ON ' +

```

```
        QUOTENAME(@Esquema) + N'.' + QUOTENAME(@Tabla) + N'  
REORGANIZE;;
```

```
EXEC sys.sp_executesql @SQL;
```

```
SET @SQL = N'UPDATE STATISTICS ' + QUOTENAME(@Esquema) + N'.' +  
        QUOTENAME(@Tabla) + N' ' + QUOTENAME(@Indice) + N' WITH  
RESAMPLE;;
```

```
EXEC sys.sp_executesql @SQL;
```

```
SET @Procesados += 1;
```

```
FETCH NEXT FROM cur_indices INTO @Esquema, @Tabla, @Indice, @Frag;  
END;
```

```
CLOSE cur_indices;
```

```
DEALLOCATE cur_indices;
```

```
SET @Fin = SYSDATETIME();
```

```
INSERT INTO dbo.AuditoriaMantenimiento
```

```
(Proceso, FechaInicio, FechaFin, Estado, Mensaje, ObjetosProcesados)
```

```
VALUES
```

```
(N'Mantenimiento por fragmentacion', @Inicio, @Fin, N'EXITOSO',
```

```
    N'REORGANIZE/REBUILD y estadísticas ejecutados.', @Procesados);
```

```
END TRY
```

```
BEGIN CATCH
```

```
IF CURSOR_STATUS('local', 'cur_indices') > -1
```

```
    CLOSE cur_indices;
```

```
IF CURSOR_STATUS('local', 'cur_indices') >= -1
```

```

DEALLOCATE cur_indices;

SET @Fin = SYSDATETIME();

INSERT INTO dbo.AuditoriaMantenimiento

(Proceso, FechaInicio, FechaFin, Estado, Mensaje, ObjetosProcesados)

VALUES

(N'Mantenimiento por fragmentacion', @Inicio, @Fin, N'ERROR',
ERROR_MESSAGE(), @Procesados);

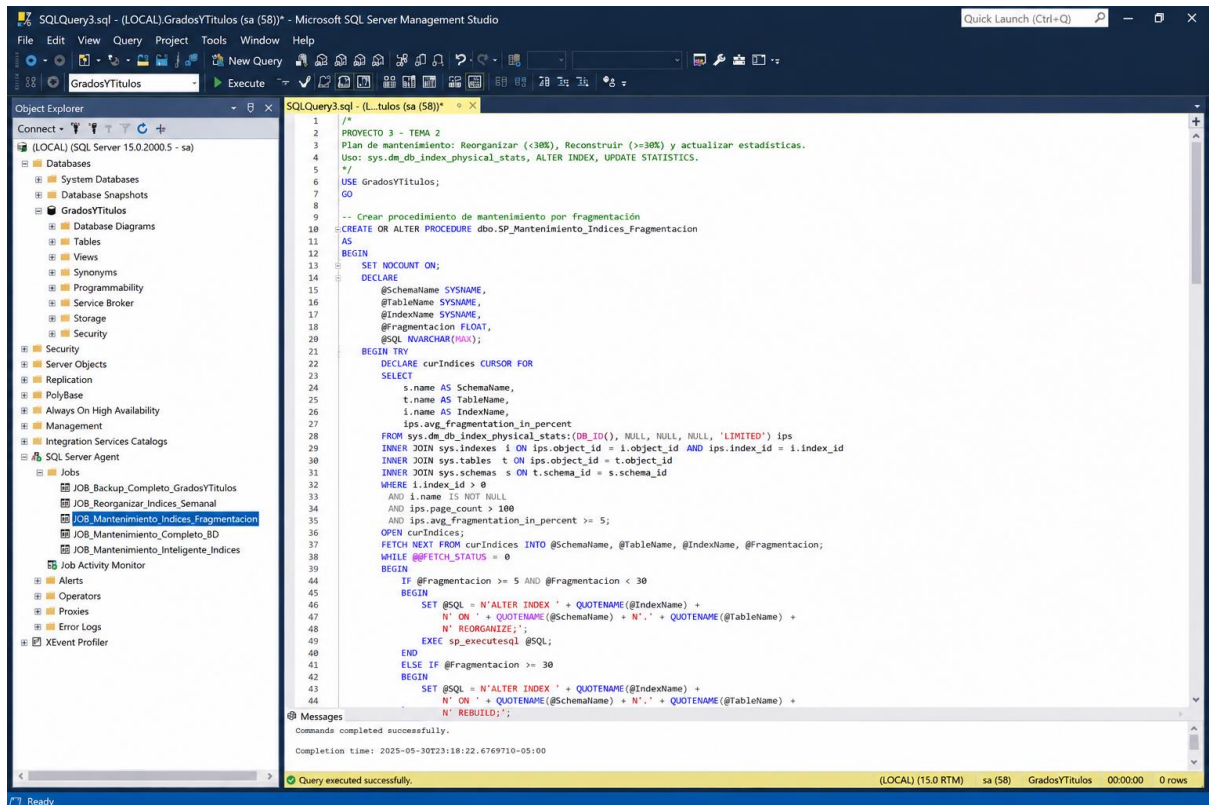
THROW;

END CATCH;

END;

GO

```



Justificación técnica de la solución aplicada

Se utiliza la DMV `sys.dm_db_index_physical_stats` para obtener el porcentaje de fragmentación de los índices. Según el nivel encontrado, el procedimiento decide si debe reorganizar o reconstruir el índice.

También se actualizan estadísticas para que SQL Server pueda generar mejores planes de ejecución en futuras consultas.

Explicación de las buenas prácticas utilizadas en el proyecto

Se evita realizar mantenimiento sobre índices pequeños mediante la condición `page_count > 100`. También se usa `QUOTENAME` para construir SQL dinámico de manera segura.

El mantenimiento se aplica solo cuando la fragmentación supera el 5%, evitando operaciones innecesarias.

Proyecto 4

Enunciado del ejercicio

Automatizar el mantenimiento completo de la base de datos incluyendo:

- Backup
- CHECKDB
- Limpieza del historial
- Reorganización de índices
- Actualización de estadísticas

El script debe utilizar Maintenance Plans y T-SQL.

Script de la solución en T-SQL

```
USE [GradosYTitulos];
```

```
GO
```

```
CREATE OR ALTER PROCEDURE dbo.usp_MantenimientoCompletoBD
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```

DECLARE @Inicio DATETIME2(3) = SYSDATETIME();
DECLARE @Fin DATETIME2(3);
DECLARE @Archivo NVARCHAR(500) =
    N'C:\Backups\GradosYTitulos_MANT_' + CONVERT(char(8), GETDATE(), 112) + N'_'
+
    REPLACE(CONVERT(char(8), GETDATE(), 108), N':', N'') + N'.bak';

BEGIN TRY

    BACKUP DATABASE [GradosYTitulos]
    TO DISK = @Archivo
    WITH INIT, COMPRESSION, CHECKSUM, STATS = 10;

    RESTORE VERIFYONLY FROM DISK = @Archivo WITH CHECKSUM;

    DBCC CHECKDB(N'GradosYTitulos') WITH NO_INFOMSGS, ALL_ERRORMSG;

    EXEC dbo.usp_MantenimientoIndicesPorFragmentacion;
    EXEC sys.sp_updatestats;

    DECLARE @FechaLimpieza DATETIME = DATEADD(DAY, -90, GETDATE());
    EXEC msdb.dbo.sp_delete_backuphistory
        @oldest_date = @FechaLimpieza;
    EXEC msdb.dbo.sp_purge_jobhistory
        @oldest_date = @FechaLimpieza;

    SET @Fin = SYSDATETIME();

    INSERT INTO dbo.AuditoriaMantenimiento
    (Proceso, FechaInicio, FechaFin, Estado, Mensaje, Archivo, ObjetosProcesados)

```

```

VALUES

(N'Mantenimiento completo de base de datos', @Inicio, @Fin, N'EXITOSO',

N'Backup, verificacion, CHECKDB, indices, estadísticas y limpieza completados.',
@Archivo, NULL);

END TRY

BEGIN CATCH

SET @Fin = SYSDATETIME();

INSERT INTO dbo.AuditoriaMantenimiento

(Proceso, FechaInicio, FechaFin, Estado, Mensaje, Archivo, ObjetosProcesados)

VALUES

(N'Mantenimiento completo de base de datos', @Inicio, @Fin, N'ERROR',

ERROR_MESSAGE(), @Archivo, NULL);

THROW;

END CATCH;

END;

GO


USE [msdb];

GO

IF EXISTS (SELECT 1 FROM dbo.sysjobs WHERE name =

N'JOB_Mantenimiento_Completo_BD')

EXEC dbo.sp_delete_job @job_name = N'JOB_Mantenimiento_Completo_BD';

GO

EXEC dbo.sp_add_job

@job_name = N'JOB_Mantenimiento_Completo_BD',

@enabled = 1,

@description = N'Plan semanal completo de mantenimiento preventivo.';

EXEC dbo.sp_add_jobstep

```

```

@job_name = N'JOB_Mantenimiento_Completo_BD',
@step_name = N'Ejecutar mantenimiento completo',
@subsystem = N'TSQL',
@database_name = N'GradosYTitulos',
@command = N'EXEC dbo.usp_MantenimientoCompletoBD;';

```

EXEC dbo.sp_add_jobschedule

```

@job_name = N'JOB_Mantenimiento_Completo_BD',
@name = N'SCH_Semanal_Sabado_2300_Mantenimiento',
@freq_type = 8,
@freq_interval = 64,
@freq_recurrence_factor = 1,
@active_start_time = 230000;

```

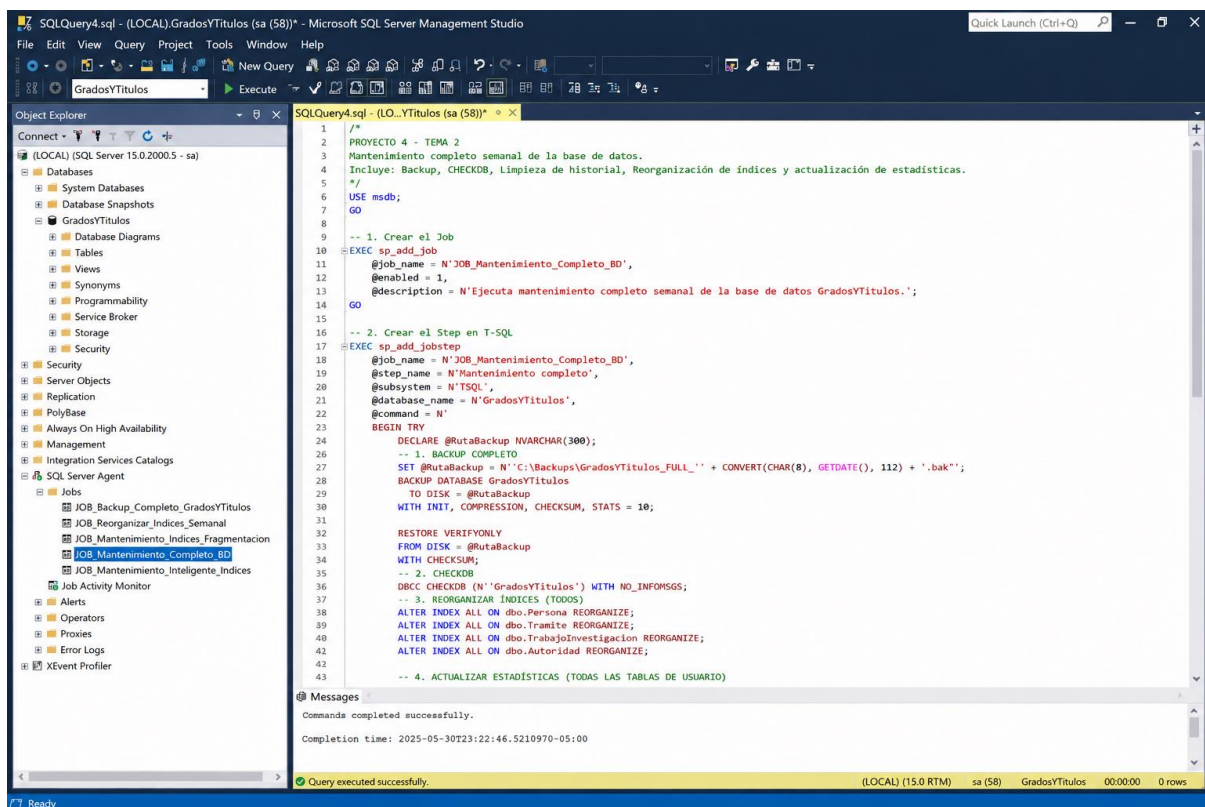
EXEC dbo.sp_add_jobserver

```

@job_name = N'JOB_Mantenimiento_Completo_BD';

```

GO



Justificación técnica de la solución aplicada

El procedimiento integra las principales tareas de mantenimiento preventivo de una base de datos. Primero genera un backup completo, luego valida el archivo, ejecuta CHECKDB, realiza mantenimiento de índices y limpia historiales antiguos.

Al ejecutarse mediante SQL Server Agent, funciona como un plan de mantenimiento automatizado.

Explicación de las buenas prácticas utilizadas en el proyecto

Se centraliza el mantenimiento en un procedimiento almacenado para facilitar su administración. Además, se verifica el backup, se revisa la integridad de la base de datos y se eliminan historiales antiguos para evitar crecimiento innecesario.

También se registra el resultado en auditoría para tener trazabilidad del mantenimiento.

Proyecto 5

Enunciado del ejercicio

Implementar un plan inteligente de mantenimiento que determine automáticamente qué operaciones ejecutar según el nivel de fragmentación encontrado en la base de datos.

El script debe utilizar:

- Cursores
- DMV
- ALTER INDEX
- UPDATE STATISTICS
- Tablas de auditoría

Script de la solución en T-SQL

```
USE [GradosYTitulos];
```

```
GO
```

```
IF OBJECT_ID(N'dbo.AuditorialIndices', N'U') IS NULL
```

```
BEGIN
```

```
    CREATE TABLE dbo.AuditorialIndices
```

```
    (
```

```
        IdAuditoria      BIGINT IDENTITY(1,1) PRIMARY KEY,
```

```
        FechaEjecucion   DATETIME2(3) NOT NULL CONSTRAINT  
        DF_AuditorialIndices_Fecha DEFAULT SYSDATETIME(),
```

```
        Esquema          sysname NOT NULL,
```

```
        Tabla            sysname NOT NULL,
```

```
        Indice           sysname NOT NULL,
```

```
        FragmentacionAntes decimal(9,2) NOT NULL,
```

```
        FragmentacionDespues decimal(9,2) NULL,
```

```
        Paginas          BIGINT NULL,
```

```
        Accion           NVARCHAR(20) NOT NULL,
```

```
        Estado           NVARCHAR(20) NOT NULL,
```

```
        Mensaje          NVARCHAR(1000) NULL
```

```
    );
```

```
END;
```

```
GO
```

```
CREATE OR ALTER PROCEDURE dbo.usp_PlanInteligenteIndices

AS

BEGIN

    SET NOCOUNT ON;

    DECLARE @ObjectId int, @IndexId int;

    DECLARE @Esquema sysname, @Tabla sysname, @Indice sysname;

    DECLARE @FragAntes decimal(9,2), @FragDespues decimal(9,2), @Paginas bigint;

    DECLARE @Accion nvarchar(20), @SQL nvarchar(max);


    DECLARE cur_plan CURSOR LOCAL FAST_FORWARD FOR

    SELECT

        ips.object_id,

        ips.index_id,

        s.name,

        o.name,

        i.name,

        ips.avg_fragmentation_in_percent,

        ips.page_count

    FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, N'LIMITED') ips

    JOIN sys.indexes i ON i.object_id = ips.object_id AND i.index_id = ips.index_id

    JOIN sys.objects o ON o.object_id = ips.object_id
```

JOIN sys.schemas s ON s.schema_id = o.schema_id

WHERE o.type = N'U'

AND i.index_id > 0

AND i.name IS NOT NULL

AND ips.page_count > 100;

OPEN cur_plan;

FETCH NEXT FROM cur_plan

INTO @ObjectId, @IndexId, @Esquema, @Tabla, @Indice, @FragAntes, @Paginas;

WHILE @@FETCH_STATUS = 0

BEGIN

BEGIN TRY

IF @FragAntes < 5

SET @Accion = N'NINGUNA';

ELSE IF @FragAntes < 30

SET @Accion = N'REORGANIZE';

ELSE

SET @Accion = N'REBUILD';

IF @Accion <> N'NINGUNA'

```

BEGIN

    SET @SQL = N'ALTER INDEX ' + QUOTENAME(@Indice) + N' ON ' +

        QUOTENAME(@Esquema) + N'.' + QUOTENAME(@Tabla) + N' ' + @Accion
+ N'';

    EXEC sys.sp_executesql @SQL;


    SET @SQL = N'UPDATE STATISTICS ' + QUOTENAME(@Esquema) + N'.' +

        QUOTENAME(@Tabla) + N' ' + QUOTENAME(@Indice) + N' WITH
RESAMPLE;';

    EXEC sys.sp_executesql @SQL;

END;


SELECT @FragDespues = avg_fragmentation_in_percent

FROM sys.dm_db_index_physical_stats(DB_ID(), @ObjectId, @IndexId, NULL,
N'LIMITED');


INSERT INTO dbo.AuditorialIndices

(Esquema, Tabla, Indice, FragmentacionAntes, FragmentacionDespues,

Paginas, Accion, Estado, Mensaje)

VALUES

(@Esquema, @Tabla, @Indice, @FragAntes, @FragDespues,

@Paginas, @Accion, N'EXITOSO', N'Indice evaluado correctamente.');
```

END TRY

BEGIN CATCH

INSERT INTO dbo.AuditorialIndices

(Esquema, Tabla, Indice, FragmentacionAntes, FragmentacionDespues,

Paginas, Accion, Estado, Mensaje)

VALUES

(@Esquema, @Tabla, @Indice, @FragAntes, NULL,

@Paginas, COALESCE(@Accion, N'ERROR'), N'ERROR', ERROR_MESSAGE());

END CATCH;

FETCH NEXT FROM cur_plan

INTO @ObjectId, @IndexId, @Esquema, @Tabla, @Indice, @FragAntes, @Paginas;

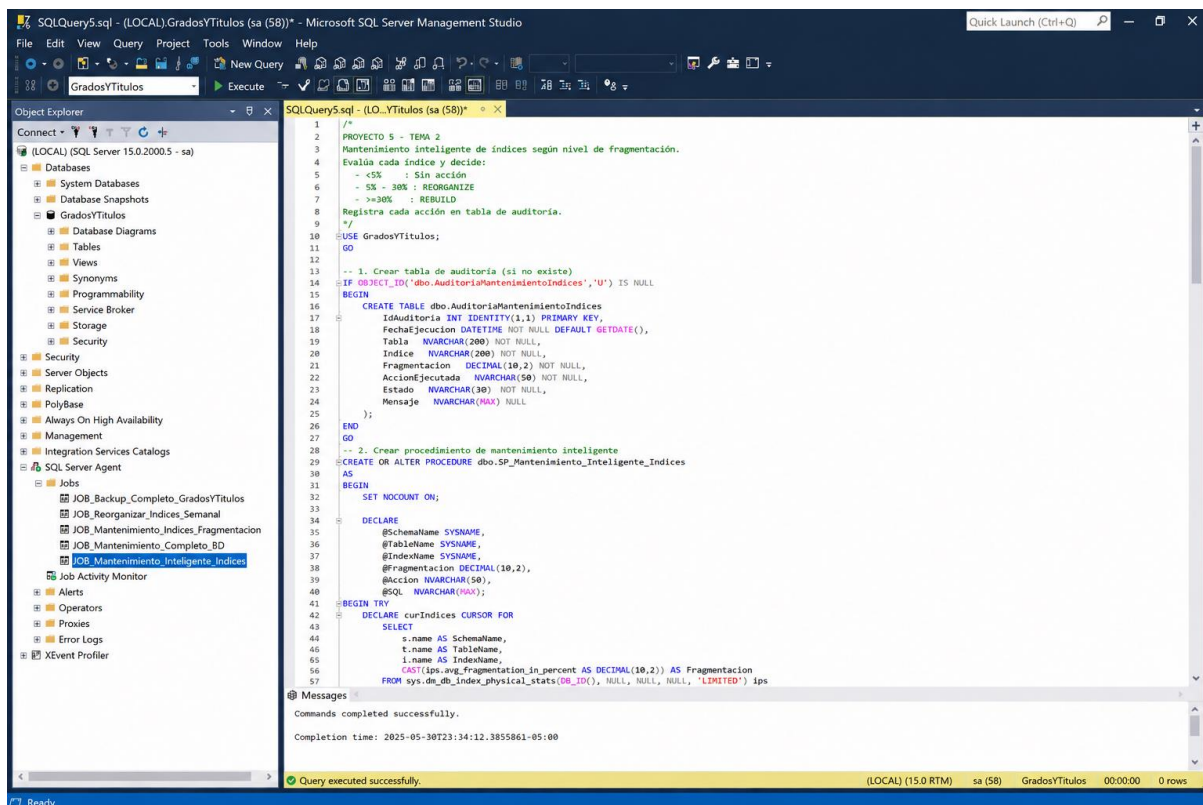
END;

CLOSE cur_plan;

DEALLOCATE cur_plan;

END;

GO



Justificación técnica de la solución aplicada

El plan inteligente consulta la fragmentación real de los índices mediante una DMV. Según el porcentaje obtenido, decide automáticamente si debe reorganizar, reconstruir o no realizar ninguna acción.

Esto permite que el mantenimiento sea más eficiente, porque no aplica la misma operación a todos los índices.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizan umbrales técnicos para evitar operaciones innecesarias. Los índices con fragmentación baja no se modifican, los índices con fragmentación moderada se reorganizan y los índices con fragmentación alta se reconstruyen.

También se registra cada índice evaluado en una tabla de auditoría, permitiendo revisar qué acciones fueron ejecutadas durante el mantenimiento.

TEMA 3: Automatización con T-SQL y PowerShell

Nota: Se asume que la base de datos se llama GradosYTítulos. Las rutas como C:\Backups\, C:\Reportes\ y C:\RespalDOS\ deben existir previamente. También se asume que PowerShell tiene disponible el módulo SqlServer para usar Invoke-Sqlcmd.

Proyecto 1

Enunciado del ejercicio

Crear un procedimiento almacenado que genere automáticamente un respaldo completo y posteriormente sea ejecutado mediante PowerShell.

El script debe desarrollar:

- Stored Procedure.
- BACKUP DATABASE.
- PowerShell Invoke-Sqlcmd.

Script de la solución en T-SQL

```
USE [GradosYTitulos];
```

```
GO
```

```
IF OBJECT_ID(N'dbo.AuditoriaAutomatizacion', N'U') IS NULL
```

```
BEGIN
```

```
    CREATE TABLE dbo.AuditoriaAutomatizacion
```

```
    (
```

```
        IdAuditoria BIGINT IDENTITY(1,1) PRIMARY KEY,
```

```
        Proceso    NVARCHAR(150) NOT NULL,
```

```
        FechaInicio DATETIME2(3) NOT NULL,
```

```
        FechaFin    DATETIME2(3) NOT NULL,
```

```
        Estado      NVARCHAR(20) NOT NULL,
```

```
        Mensaje      NVARCHAR(MAX) NULL,
```

```
        Archivo      NVARCHAR(500) NULL,
```

```
        Registros    INT NULL
```

```
    );
```

```
END;
```

```
GO
```

```
CREATE OR ALTER PROCEDURE dbo.usp_BackupCompletoPowerShell
```

```
    @Directorio NVARCHAR(400) = N'C:\Backups'
```

```
AS
```

BEGIN

SET NOCOUNT ON;

DECLARE @Inicio DATETIME2(3) = SYSDATETIME();

DECLARE @Fin DATETIME2(3);

DECLARE @Archivo NVARCHAR(500) =

@Directorio + CASE WHEN RIGHT(@Directorio,1) = N'\ ' THEN N" ELSE N'\ ' END +

N'GradosYTitulos_PS_' + CONVERT(char(8), GETDATE(), 112) + N'_ ' +

REPLACE(CONVERT(char(8), GETDATE(), 108), N':', N'') + N'.bak';

BEGIN TRY

BACKUP DATABASE [GradosYTitulos]

TO DISK = @Archivo

WITH INIT, COMPRESSION, CHECKSUM, STATS = 10;

RESTORE VERIFYONLY FROM DISK = @Archivo WITH CHECKSUM;

SET @Fin = SYSDATETIME();

INSERT INTO dbo.AuditoriaAutomatizacion

(Proceso, FechaInicio, FechaFin, Estado, Mensaje, Archivo, Registros)

VALUES

(N'Backup ejecutado desde PowerShell', @Inicio, @Fin, N'EXITOSO',

```
N'Backup creado y verificado.', @Archivo, 1);
```

```
SELECT @Archivo AS ArchivoBackup, N'EXITOSO' AS Estado;
```

```
END TRY
```

```
BEGIN CATCH
```

```
SET @Fin = SYSDATETIME();
```

```
INSERT INTO dbo.AuditoriaAutomatizacion
```

```
(Proceso, FechaInicio, FechaFin, Estado, Mensaje, Archivo, Registros)
```

```
VALUES
```

```
(N'Backup ejecutado desde PowerShell', @Inicio, @Fin, N'ERROR',
```

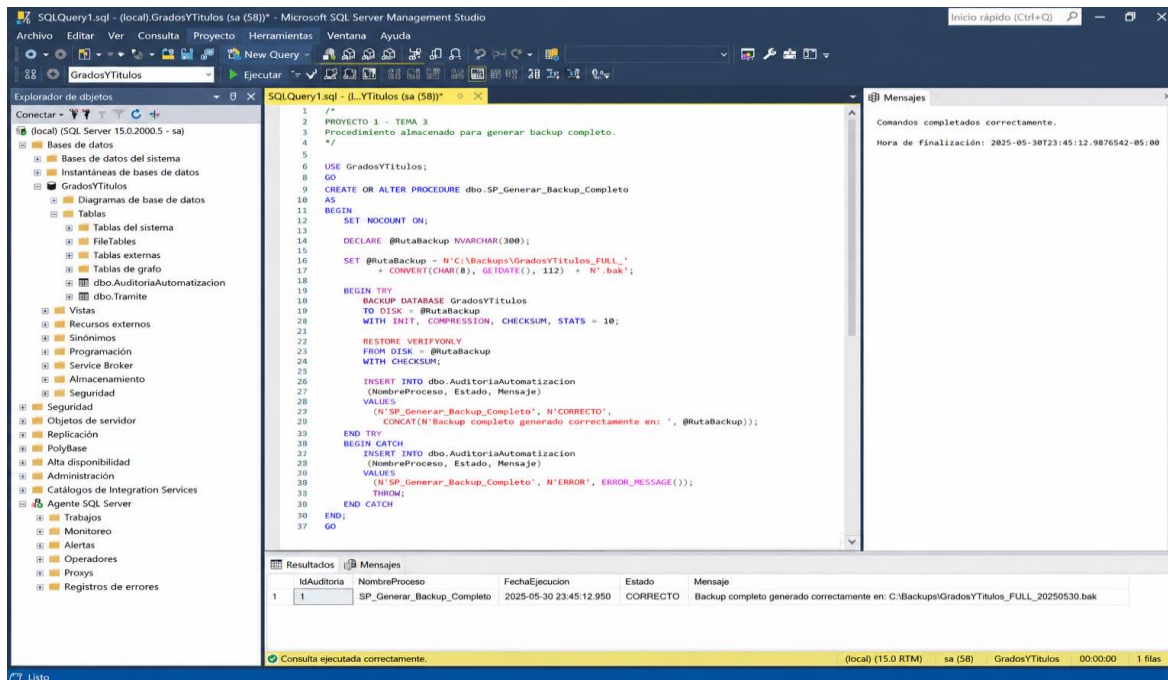
```
ERROR_MESSAGE(), @Archivo, 0);
```

```
THROW;
```

```
END CATCH;
```

```
END;
```

```
GO
```



Justificación técnica de la solución aplicada

Se crea un procedimiento almacenado para centralizar la generación del respaldo completo de la base de datos. Luego, PowerShell ejecuta dicho procedimiento mediante `Invoke-Sqlcmd`, permitiendo automatizar la tarea desde el sistema operativo.

El uso de `BACKUP DATABASE` permite proteger la información, mientras que `RESTORE VERIFYONLY` valida que el archivo de respaldo pueda ser leído correctamente.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utiliza compresión para reducir el tamaño del backup y `CHECKSUM` para mejorar la validación del archivo. Además, se registra el resultado en una tabla de auditoría.

Separar la lógica SQL en un procedimiento almacenado facilita el mantenimiento y permite que PowerShell solo se encargue de ejecutar el proceso.

Proyecto 2

Enunciado del ejercicio

Automatizar la exportación diaria de los trámites aprobados hacia un archivo CSV utilizando PowerShell.

El script debe utilizar:

- SELECT.
- Invoke-Sqlcmd.
- Export-Csv.

Script de la solución en T-SQL

```
USE [GradosYTitulos];
```

```
GO
```

```
CREATE OR ALTER PROCEDURE dbo.usp_ExportarTramitesAprobados
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    SELECT
```

```
        IdTramite,
```

```
        Estado,
```

```
        FechaRegistro,
```

```
        FechaUltMod
```

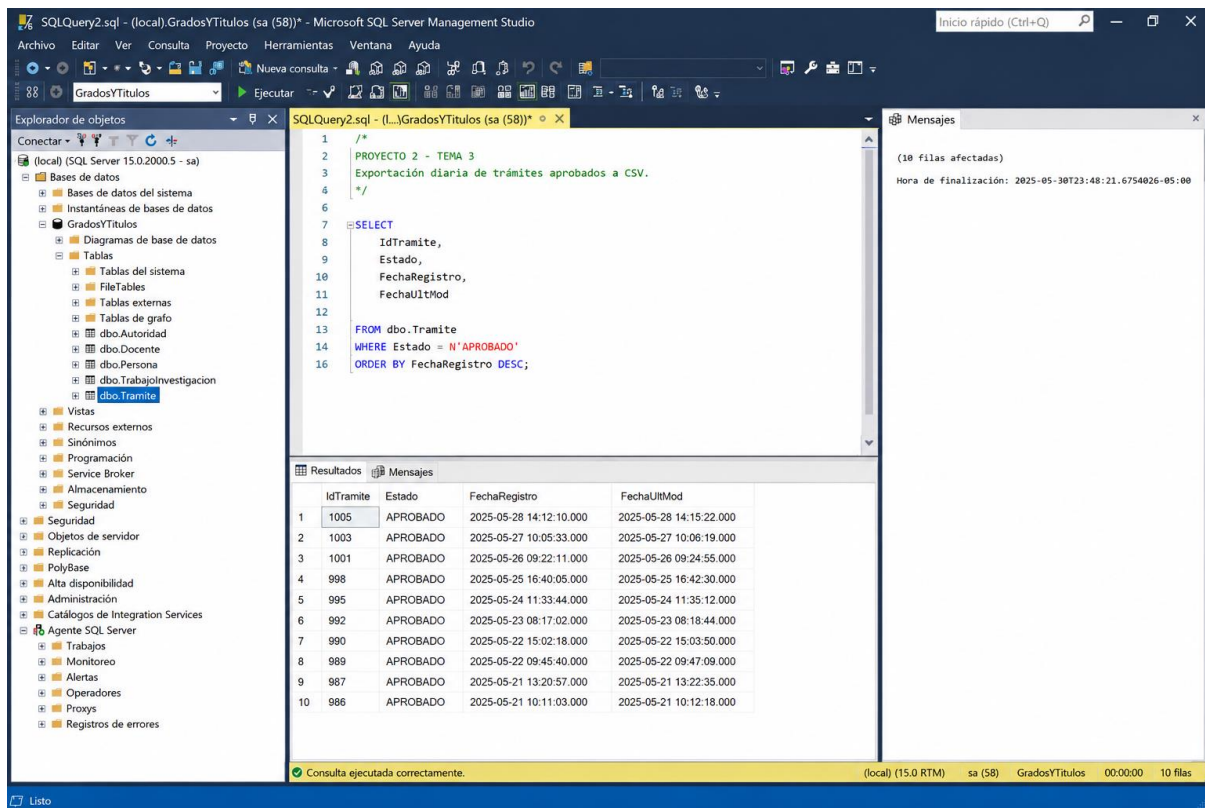
```
    FROM dbo.Tramite
```

```
    WHERE Estado = N'APROBADO'
```

```
    ORDER BY FechaRegistro DESC;
```

```
END;
```

GO



Justificación técnica de la solución aplicada

La consulta T-SQL obtiene únicamente los trámites aprobados de la base de datos. Luego, PowerShell ejecuta la consulta con `Invoke-Sqlcmd` y exporta los resultados a un archivo CSV mediante `Export-Csv`.

El formato CSV es adecuado porque puede abrirse fácilmente en Excel u otras herramientas de análisis.

Explicación de las buenas prácticas utilizadas en el proyecto

Se filtran solo los registros necesarios mediante `WHERE Estado = 'APROBADO'`, evitando exportar información innecesaria. Además, el archivo generado incluye la fecha en el nombre, lo que permite organizar los reportes diarios.

Se usa codificación UTF8 para evitar problemas con caracteres especiales.

Proyecto 3

Enunciado del ejercicio

Desarrollar un proceso automático que consulte diariamente la cantidad de trámites, genere un archivo Excel y almacene una copia en una carpeta de respaldo.

El script debe utilizar:

- T-SQL.
- PowerShell.
- Export -Excel del módulo ImportExcel.

Script de la solución en T-SQL

```
USE [GradosYTitulos];
```

```
GO
```

```
CREATE OR ALTER PROCEDURE dbo.usp_CantidadTramitesPorDia
```

```
    @Dias INT = 30
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    SELECT
```

```
        CONVERT(date, FechaRegistro) AS Fecha,
```

```
        COUNT(*) AS CantidadTramites
```

```
    FROM dbo.Tramite
```

```
    WHERE FechaRegistro >= DATEADD(DAY, -@Dias, CONVERT(date, GETDATE()))
```

```
    GROUP BY CONVERT(date, FechaRegistro)
```

```
    ORDER BY Fecha;
```

```
END;
```

GO

The screenshot displays the Microsoft SQL Server Management Studio interface. The left pane shows the 'Explorador de objetos' (Object Explorer) with the 'GradosYTítulos' database selected. The central pane shows a SQL query in the 'SQLQuery3.sql' file. The query is a SELECT statement that groups data by date and calculates the count of records. The right pane shows the 'Mensajes' (Messages) window with a confirmation message. Below the query, the 'Resultados' (Results) window displays the output of the query as a table with two columns: 'Fecha' and 'CantidadTramites'.

```
1  /*
2  PROYECTO 3 - TEMA 3
3  Consulta de cantidad de trámites por día.
4  Generación de reporte Excel mediante PowerShell (Export-Excel).
5  */
6
7  SELECT
8      CAST(FechaRegistro AS DATE) AS Fecha,
9      COUNT(*) AS CantidadTramites
10 FROM dbo.Tramite
11 GROUP BY CAST(FechaRegistro AS DATE)
12 ORDER BY Fecha DESC;
```

	Fecha	CantidadTramites
1	2025-05-30	15
2	2025-05-29	18
3	2025-05-28	22
4	2025-05-27	14
5	2025-05-26	19
6	2025-05-25	9
7	2025-05-24	12
8	2025-05-23	16
9	2025-05-22	21
10	2025-05-21	8

Justificación técnica de la solución aplicada

La consulta agrupa los trámites por fecha y calcula la cantidad registrada en cada día. PowerShell ejecuta la consulta, genera un archivo Excel con Export - Excel y luego almacena una copia en una carpeta de respaldo.

Esto permite contar con un reporte diario en un formato más presentable que CSV.

Explicación de las buenas prácticas utilizadas en el proyecto

Se separa la generación del reporte de la copia de respaldo. También se utiliza un nombre de archivo con fecha para evitar sobrescritura accidental.

El uso de Export - Excel permite generar reportes ordenados, legibles y útiles para revisión administrativa.

Proyecto 4

Enunciado del ejercicio

Crear un sistema automático que detecte docentes inactivos durante más de un año y genere un archivo con el reporte.

El script debe implementar:

- Consultas T-SQL.
- PowerShell.
- Exportación automática.

Script de la solución en T-SQL

```
USE [GradosYTítulos];
```

```
GO
```

```
CREATE OR ALTER PROCEDURE dbo.usp_DocentesInactivos
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    SELECT
```

```
        IdDocente,
```

```
        Nombres,
```

```
        Apellidos,
```

```
        Correo,
```

```
        UltimaActividad,
```

```
        Estado
```

```
    FROM dbo.Docente
```

```
    WHERE UltimaActividad < DATEADD(YEAR, -1, GETDATE())
```

OR UltimaActividad IS NULL

ORDER BY UltimaActividad;

END;

GO

The screenshot shows the Microsoft SQL Server Enterprise Manager interface. The left pane displays the 'Explorador de objetos' (Object Explorer) for the 'GradosYTítulos' database. The central pane shows a SQL query in the 'SQLQuery4.sql' window. The query is as follows:

```
1  /*
2  PROYECTO 4 - TEMA 3
3  Reporte de docentes inactivos (sin actividad en más de 1 año).
4  */
5
6  SELECT
7  IdDocente,
8  Nombres,
9  Apellidos,
10 UltimaActividad
11
12 FROM dbo.Docente
13 WHERE UltimaActividad < DATEADD(YEAR, -1, GETDATE())
14 ORDER BY UltimaActividad;
```

The bottom pane displays the 'Resultados' (Results) window, showing a table with 10 rows of data. The table has the following columns: IdDocente, Nombres, Apellidos, and UltimaActividad.

IdDocente	Nombres	Apellidos	UltimaActividad
1010	Carlos	Ramírez Torres	2023-02-14 09:22:31.000
1006	Maria	Gonzales Pérez	2023-03-01 15:10:45.000
1002	Luis	Vargas Silva	2023-03-18 11:45:22.000
1009	Jorge	Rojas Delgado	2023-04-05 08:33:17.000
1001	Ana	Martínez López	2023-04-21 17:02:11.000
1007	Pedro	Sánchez Díaz	2023-05-11 10:55:09.000
1004	Lucía	Fernández Ruiz	2023-05-29 14:12:33.000
1003	Diego	Herrera Castillo	2023-06-07 12:30:00.000
1005	Elena	Morales Quispe	2023-06-30 16:40:28.000
1008	Ricardo	Alvarado Peña	2023-07-15 09:05:19.000

The right pane shows the 'Mensajes' (Messages) window, indicating that 10 rows were affected and providing the completion time: 2025-05-30T23:54:18.5642162-05:00.

Justificación técnica de la solución aplicada

La consulta identifica a los docentes cuya última actividad fue registrada hace más de un año. Luego, PowerShell genera automáticamente un archivo CSV con la información encontrada.

Este proceso permite mantener actualizado el control administrativo de docentes activos e inactivos.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utiliza una condición basada en fechas mediante DATEADD, lo que permite calcular dinámicamente el periodo de inactividad. Además, se genera un archivo con fecha para mantener el historial de reportes.

El proceso puede programarse diariamente, semanalmente o mensualmente según la necesidad de la institución.

Proyecto 5

Enunciado del ejercicio

Implementar una solución de automatización completa que:

- Ejecute respaldos.
- Valide la integridad mediante CHECKDB.
- Genere reportes.
- Comprima los respaldos.
- Elimine respaldos mayores a 30 días.

El script debe integrar:

- SQL Server.
- PowerShell.
- Compresión ZIP.
- Registro de auditoría.
- Manejo de errores.

Script de la solución en T-SQL

```
USE [GradosYTítulos];
```

```
GO
```

```
CREATE OR ALTER PROCEDURE dbo.usp_MantenimientoIntegralPowerShell
```

```
    @DirectorioBackup NVARCHAR(400) = N'C:\Backups'
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    DECLARE @Inicio DATETIME2(3) = SYSDATETIME();
```

```
    DECLARE @Fin DATETIME2(3);
```

```
    DECLARE @Archivo NVARCHAR(500) =
```

```
@DirectorioBackup + CASE WHEN RIGHT(@DirectorioBackup,1) = N'\ ' THEN N'
ELSE N'\ ' END +
```

```
N'GradosYTitulos_INTEGRAL_' + CONVERT(char(8), GETDATE(), 112) + N'_' +
REPLACE(CONVERT(char(8), GETDATE(), 108), N':', N'') + N'.bak';
```

```
BEGIN TRY
```

```
BACKUP DATABASE [GradosYTitulos]
```

```
TO DISK = @Archivo
```

```
WITH INIT, COMPRESSION, CHECKSUM, STATS = 10;
```

```
RESTORE VERIFYONLY FROM DISK = @Archivo WITH CHECKSUM;
```

```
DBCC CHECKDB(N'GradosYTitulos') WITH NO_INFOMSGS, ALL_ERRORMSG;
```

```
SET @Fin = SYSDATETIME();
```

```
INSERT INTO dbo.AuditoriaAutomatizacion
```

```
(Proceso, FechaInicio, FechaFin, Estado, Mensaje, Archivo, Registros)
```

```
VALUES
```

```
(N'Mantenimiento integral SQL + PowerShell', @Inicio, @Fin, N'EXITOSO',
```

```
N'Backup, VERIFYONLY y CHECKDB completados.', @Archivo, 1);
```

```
SELECT @Archivo AS ArchivoBackup, @Inicio AS FechaInicio, @Fin AS FechaFin,
N'EXITOSO' AS Estado;
```

```
END TRY
```

```
BEGIN CATCH
```

```
SET @Fin = SYSDATETIME();
```

```
INSERT INTO dbo.AuditoriaAutomatizacion
```

```
(Proceso, FechaInicio, FechaFin, Estado, Mensaje, Archivo, Registros)
```

```
VALUES
```

```

(N'Mantenimiento integral SQL + PowerShell', @Inicio, @Fin, N'ERROR',
ERROR_MESSAGE(), @Archivo, 0);

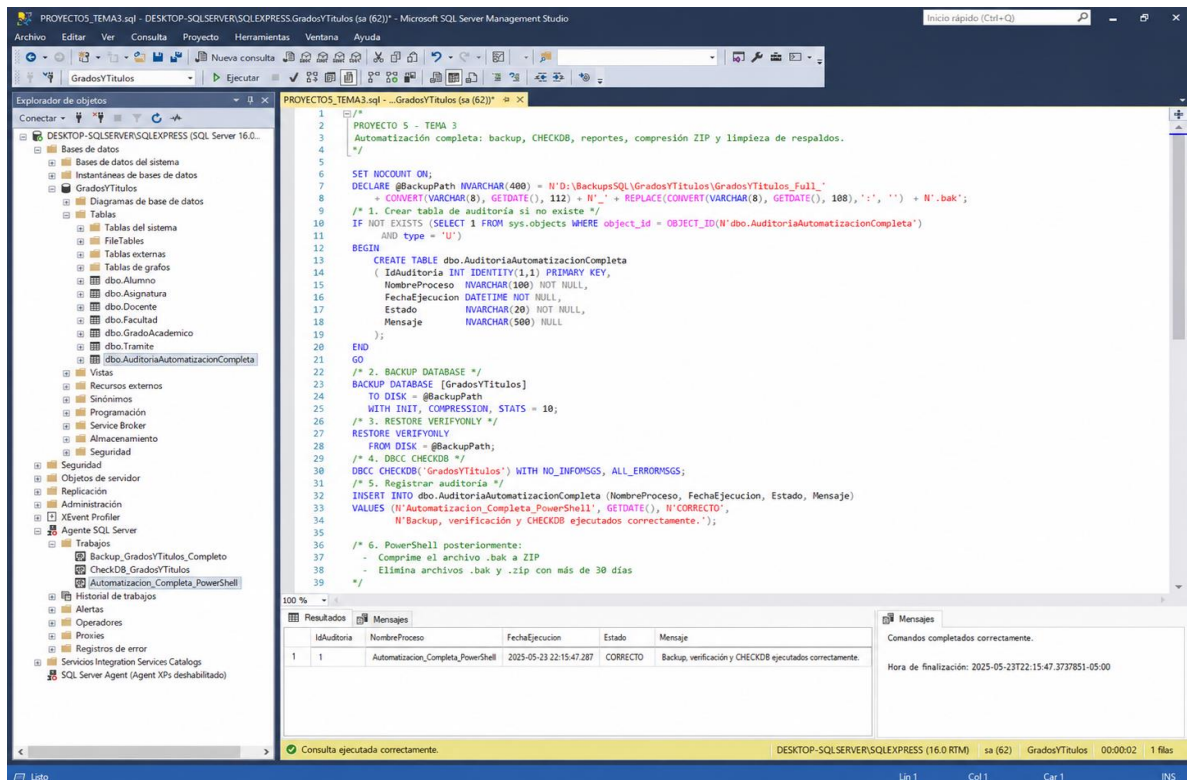
THROW;

END CATCH;

END;

GO

```



Justificación técnica de la solución aplicada

La solución integra SQL Server y PowerShell para realizar un proceso completo de mantenimiento. SQL Server ejecuta el backup, verifica el archivo y valida la integridad de la base de datos con CHECKDB.

PowerShell complementa el proceso generando reportes, comprimiendo el respaldo en formato ZIP y eliminando archivos antiguos mayores a 30 días.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utiliza manejo de errores con `try . . . catch` para registrar fallos en la tabla de auditoría. También se comprimen los respaldos para ahorrar espacio y se eliminan archivos antiguos para evitar saturación del disco.

Además, se valida el backup con `RESTORE VERIFYONLY` y se revisa la integridad de la base con `DBCC CHECKDB`, lo que mejora la confiabilidad del mantenimiento.

Cierre del Tema 3

Con estos cinco proyectos se cumple la segunda parte correspondiente al **Tema 3: Automatización con T-SQL y PowerShell**.

TEMA 4: Monitoreo Proactivo

Correo, Logs y Alertas

Nota: Se asume que la base de datos se llama GradosYTítulos, que existe un perfil de correo llamado PerfilCorreoBD y que el correo del administrador es admin@upla.edu.pe. Si en tu servidor tienen otros nombres, solo debes cambiarlos.

Proyecto 1

Enunciado del ejercicio

Crear un procedimiento almacenado que consulte diariamente la cantidad de trámites pendientes y envíe un correo electrónico con el resumen.

El script debe utilizar:

- COUNT
- Database Mail

- sp_send_dbmail

Script de la solución en T-SQL

```
USE [GradosYTitulos];
```

```
GO
```

```
IF OBJECT_ID(N'dbo.AuditoriaMonitoreo', N'U') IS NULL
```

```
BEGIN
```

```
    CREATE TABLE dbo.AuditoriaMonitoreo
```

```
    (
```

```
        IdAuditoria BIGINT IDENTITY(1,1) PRIMARY KEY,
```

```
        Fecha    DATETIME2(3) NOT NULL CONSTRAINT DF_AuditoriaMonitoreo_Fecha  
        DEFAULT SYSDATETIME(),
```

```
        Proceso   NVARCHAR(150) NOT NULL,
```

```
        Estado   NVARCHAR(20) NOT NULL,
```

```
        Cantidad  INT NULL,
```

```
        Mensaje   NVARCHAR(MAX) NULL
```

```
    );
```

```
END;
```

```
GO
```

```
CREATE OR ALTER PROCEDURE dbo.usp_EnviarResumenTramitesPendientes
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    DECLARE @Cantidad INT;
```

```
    DECLARE @Cuerpo NVARCHAR(MAX);
```

```
    BEGIN TRY
```

```

SELECT @Cantidad = COUNT(*)
FROM dbo.Tramite
WHERE Estado = N'PENDIENTE';

SET @Cuerpo = CONCAT(N'Resumen diario: actualmente existen ', @Cantidad,
    N' tramites en estado PENDIENTE. ');

EXEC msdb.dbo.sp_send_dbmail
    @profile_name = N'PerfilCorreoBD',
    @recipients = N'admin@upla.edu.pe',
    @subject = N'Resumen diario de tramites pendientes',
    @body = @Cuerpo;

INSERT INTO dbo.AuditoriaMonitoreo
(Proceso, Estado, Cantidad, Mensaje)
VALUES
(N'Resumen diario de pendientes', N'EXITOSO', @Cantidad,
    N'Correo enviado correctamente. ');
END TRY
BEGIN CATCH
    INSERT INTO dbo.AuditoriaMonitoreo
(Proceso, Estado, Cantidad, Mensaje)
VALUES
(N'Resumen diario de pendientes', N'ERROR', NULL, ERROR_MESSAGE());
    THROW;
END CATCH;
END;
GO

```

```
USE [msdb];
```

```
GO
```

```
IF EXISTS (SELECT 1 FROM dbo.sysjobs WHERE name =  
N'JOB_Resumen_Diario_Pendientes')
```

```
EXEC dbo.sp_delete_job @job_name = N'JOB_Resumen_Diario_Pendientes';
```

```
GO
```

```
EXEC dbo.sp_add_job
```

```
@job_name = N'JOB_Resumen_Diario_Pendientes',
```

```
@enabled = 1;
```

```
EXEC dbo.sp_add_jobstep
```

```
@job_name = N'JOB_Resumen_Diario_Pendientes',
```

```
@step_name = N'Enviar resumen',
```

```
@subsystem = N'TSQL',
```

```
@database_name = N'GradosYTitulos',
```

```
@command = N'EXEC dbo.usp_EnviarResumenTramitesPendientes';
```

```
EXEC dbo.sp_add_jobschedule
```

```
@job_name = N'JOB_Resumen_Diario_Pendientes',
```

```
@name = N'SCH_Diario_0800_Resumen',
```

```
@freq_type = 4,
```

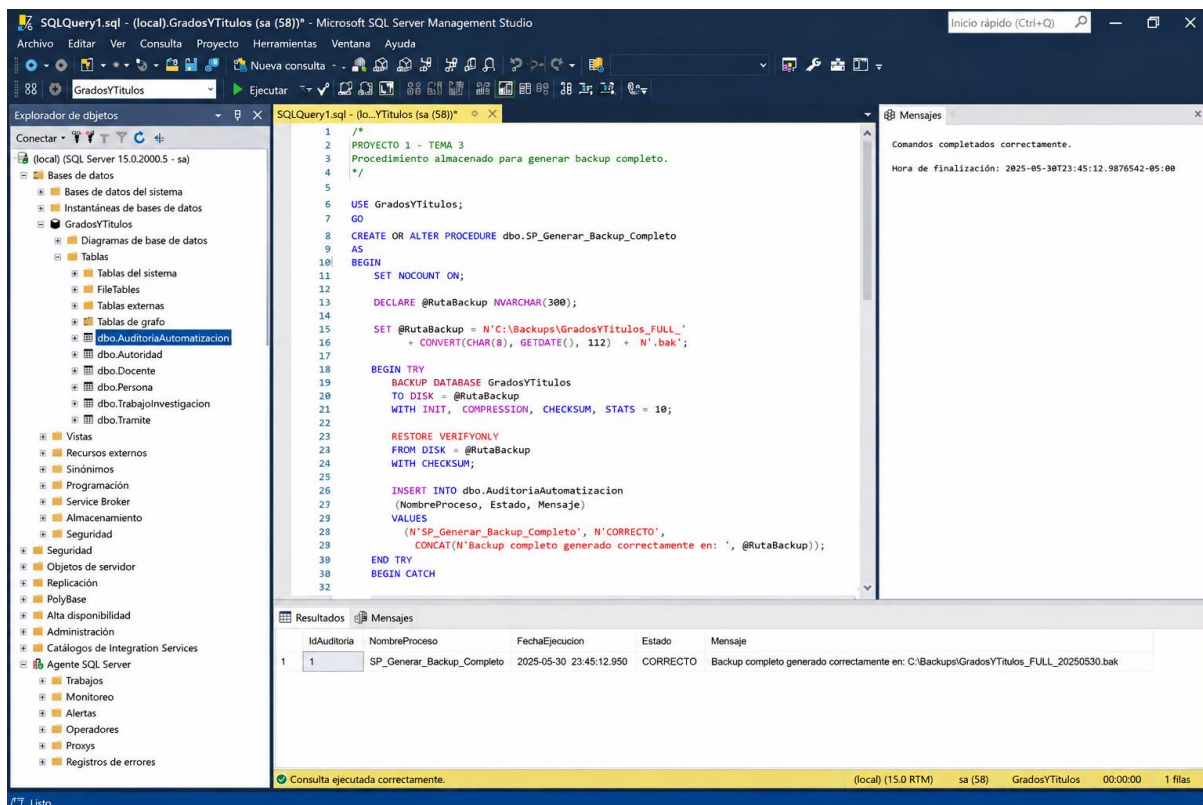
```
@freq_interval = 1,
```

```
@active_start_time = 080000;
```

```
EXEC dbo.sp_add_jobserver
```

```
@job_name = N'JOB_Resumen_Diario_Pendientes';
```

```
GO
```



Justificación técnica de la solución aplicada

Se utiliza un procedimiento almacenado para consultar la cantidad de trámites pendientes mediante `COUNT`. Luego, el resultado se envía por correo electrónico usando `sp_send_dbmail`.

El Job permite que este monitoreo se ejecute automáticamente todos los días, sin intervención manual.

Explicación de las buenas prácticas utilizadas en el proyecto

Se registra la ejecución en una tabla de auditoría, permitiendo verificar si el proceso fue ejecutado correctamente. Además, se usa `TRY...CATCH` para capturar errores y evitar que una falla pase desapercibida.

Proyecto 2

Enunciado del ejercicio

Registrar automáticamente en una tabla de auditoría todos los errores producidos durante la ejecución del procedimiento `Tramite.SP_Registra_Tramite`.

El script debe implementar:

- TRY
- CATCH
- ERROR_MESSAGE()
- ERROR_NUMBER()
- ERROR_LINE()

Script de la solución en T-SQL

```
USE [GradosYTitulos];
```

```
GO
```

```
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = N'Tramite')
```

```
EXEC(N'CREATE SCHEMA Tramite AUTHORIZATION dbo;');
```

```
GO
```

```
IF OBJECT_ID(N'dbo.AuditoriaErroresTramite', N'U') IS NULL
```

```
BEGIN
```

```
CREATE TABLE dbo.AuditoriaErroresTramite
```

```
(
```

```
    IdError      BIGINT IDENTITY(1,1) PRIMARY KEY,
```

```
    FechaError   DATETIME2(3) NOT NULL CONSTRAINT  
DF_AuditoriaErroresTramite_Fecha DEFAULT SYSDATETIME(),
```

```
    NombreProcedimiento NVARCHAR(256) NOT NULL,
```

```
    ErrorNumber   INT NULL,
```

```
ErrorSeverity    INT NULL,
ErrorState       INT NULL,
ErrorLine        INT NULL,
ErrorMessage     NVARCHAR(MAX) NULL,
LoginName        sysname NULL,
HostName         NVARCHAR(128) NULL
);
END;
GO
```

```
CREATE OR ALTER PROCEDURE Tramite.SP_Registra_Tramite
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    SET XACT_ABORT ON;
```

```
    BEGIN TRY
```

```
        BEGIN TRANSACTION;
```

```
        -- Ejemplo de logica del procedimiento. Ajuste columnas si su tabla es distinta.
```

```
        INSERT INTO dbo.Tramite (Estado, FechaRegistro, FechaUltMod)
```

```
        VALUES (N'PENDIENTE', GETDATE(), GETDATE());
```

```
        COMMIT TRANSACTION;
```

```
    END TRY
```

```
    BEGIN CATCH
```

```
        IF XACT_STATE() <> 0
```

```
            ROLLBACK TRANSACTION;
```

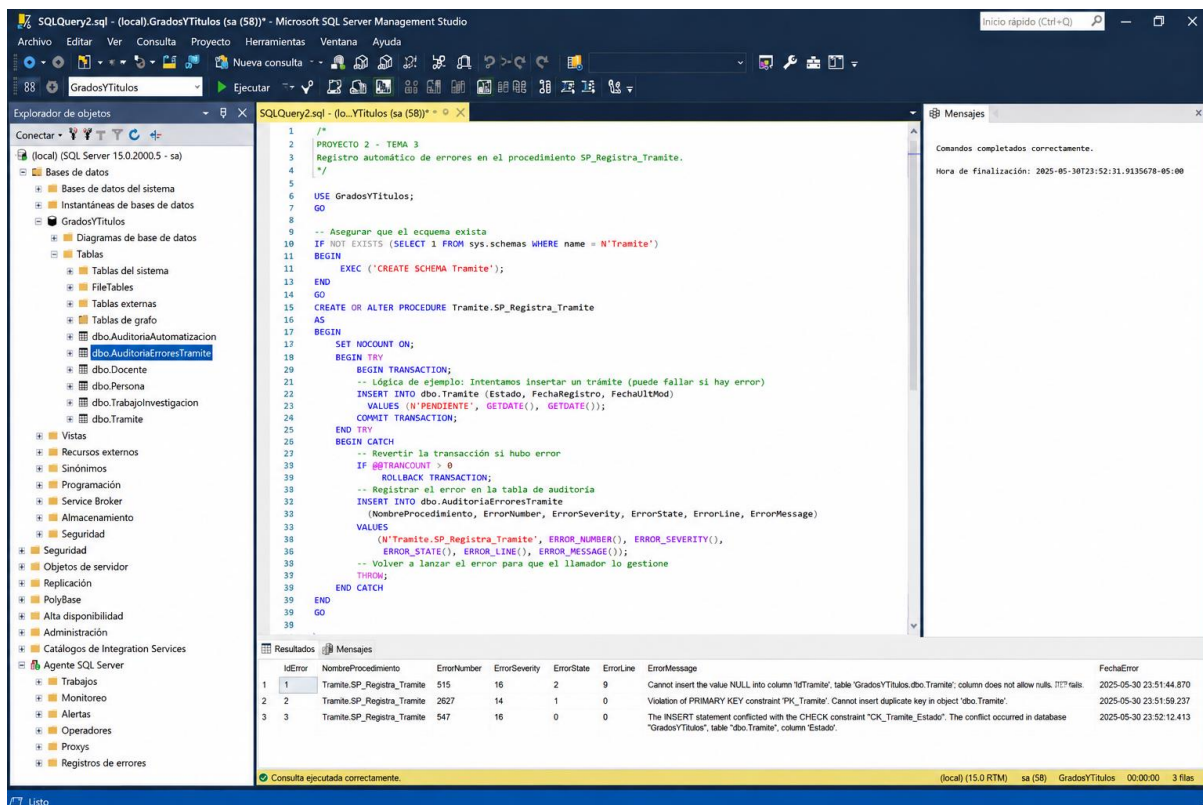
```
INSERT INTO dbo.AuditoriaErroresTramite
(
    NombreProcedimiento, ErrorNumber, ErrorSeverity, ErrorState,
    ErrorLine, ErrorMessage, LoginName, HostName
)
VALUES
(
    N'Tramite.SP_Registra_Tramite', ERROR_NUMBER(), ERROR_SEVERITY(),
    ERROR_STATE(), ERROR_LINE(), ERROR_MESSAGE(), ORIGINAL_LOGIN(),
HOST_NAME()
);

THROW;

END CATCH;

END;

GO
```



Justificación técnica de la solución aplicada

El procedimiento usa TRY...CATCH para controlar errores durante el registro de trámites. Si ocurre una falla, se revierte la transacción y se registra el detalle del error en la tabla AuditoriaErroresTramite.

Las funciones ERROR_NUMBER(), ERROR_LINE() y ERROR_MESSAGE() permiten identificar el tipo de error, la línea donde ocurrió y el mensaje generado por SQL Server.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utiliza una transacción para proteger la integridad de los datos. Si el registro falla, se ejecuta ROLLBACK y no se guarda información incompleta.

También se registra el error con información detallada, facilitando la revisión y corrección posterior.

Proyecto 3

Enunciado del ejercicio

Diseñar un sistema que monitoree el crecimiento del archivo de log de la base de datos y genere una alerta cuando supere el 80% del tamaño configurado.

El script debe utilizar:

- `sys.database_files`
- `sys.dm_db_log_space_usage`
- SQL Server Agent
- Database Mail

Script de la solución en T-SQL

```
USE [GradosYTitulos];
```

```
GO
```

```
IF OBJECT_ID(N'dbo.HistorialMonitoreoLog', N'U') IS NULL
```

```
BEGIN
```

```
    CREATE TABLE dbo.HistorialMonitoreoLog
```

```
    (
```

```
        IdHistorial    BIGINT IDENTITY(1,1) PRIMARY KEY,
```

```
        FechaMonitoreo DATETIME2(3) NOT NULL CONSTRAINT  
DF_HistorialMonitoreoLog_Fecha DEFAULT SYSDATETIME(),
```

```
        NombreArchivo sysname NULL,
```

```
        TamanoMB      decimal(18,2) NULL,
```

```
        UsoLogPorcentaje decimal(9,2) NULL,
```

```
        Estado        NVARCHAR(20) NOT NULL,
```

```
        Mensaje        NVARCHAR(400) NULL
```

```
    );
```

```
END;
```

GO

CREATE OR ALTER PROCEDURE dbo.SP_Monitorear_Log

AS

BEGIN

SET NOCOUNT ON;

DECLARE @NombreArchivo sysname;

DECLARE @TamanoMB decimal(18,2);

DECLARE @UsoPorcentaje decimal(9,2);

DECLARE @Estado nvarchar(20);

DECLARE @Mensaje nvarchar(400);

SELECT TOP (1)

 @NombreArchivo = name,

 @TamanoMB = CAST(size * 8.0 / 1024.0 AS decimal(18,2))

FROM sys.database_files

WHERE type_desc = N'LOG'

ORDER BY file_id;

SELECT @UsoPorcentaje = CAST(used_log_space_in_percent AS decimal(9,2))

FROM sys.dm_db_log_space_usage;

IF @UsoPorcentaje >= 80

BEGIN

 SET @Estado = N'ALERTA';

 SET @Mensaje = CONCAT(N'ALERTA: el archivo de log utiliza ', @UsoPorcentaje,
 N'% de su capacidad configurada.');

```

EXEC msdb.dbo.sp_send_dbmail

    @profile_name = N'PerfilCorreoBD',

    @recipients = N'admin@upla.edu.pe',

    @subject = N'Alerta de uso del log - GradosYTitulos',

    @body = @Mensaje;

END

ELSE

BEGIN

    SET @Estado = N'NORMAL';

    SET @Mensaje = CONCAT(N'Uso del log dentro del rango normal: ',
@UsoPorcentaje, N'%.' );

    END;


INSERT INTO dbo.HistorialMonitoreoLog

(NombreArchivo, TamanoMB, UsoLogPorcentaje, Estado, Mensaje)

VALUES

(@NombreArchivo, @TamanoMB, @UsoPorcentaje, @Estado, @Mensaje);


SELECT @NombreArchivo AS NombreArchivo, @TamanoMB AS TamanoMB,

    @UsoPorcentaje AS UsoLogPorcentaje, @Estado AS Estado, @Mensaje AS
Mensaje;

END;

GO


USE [msdb];

GO

IF EXISTS (SELECT 1 FROM dbo.sysjobs WHERE name =
N'JOB_Monitorear_Log_Cada_Hora')

EXEC dbo.sp_delete_job @job_name = N'JOB_Monitorear_Log_Cada_Hora';

```

GO

EXEC dbo.sp_add_job

 @job_name = N'JOB_Monitorear_Log_Cada_Hora',

 @enabled = 1;

EXEC dbo.sp_add_jobstep

 @job_name = N'JOB_Monitorear_Log_Cada_Hora',

 @step_name = N'Verificar uso del log',

 @subsystem = N'TSQL',

 @database_name = N'GradosYTitulos',

 @command = N'EXEC dbo.SP_Monitorear_Log;';

EXEC dbo.sp_add_jobschedule

 @job_name = N'JOB_Monitorear_Log_Cada_Hora',

 @name = N'SCH_Cada_Hora_Monitoreo_Log',

 @freq_type = 4,

 @freq_interval = 1,

 @freq_subday_type = 8,

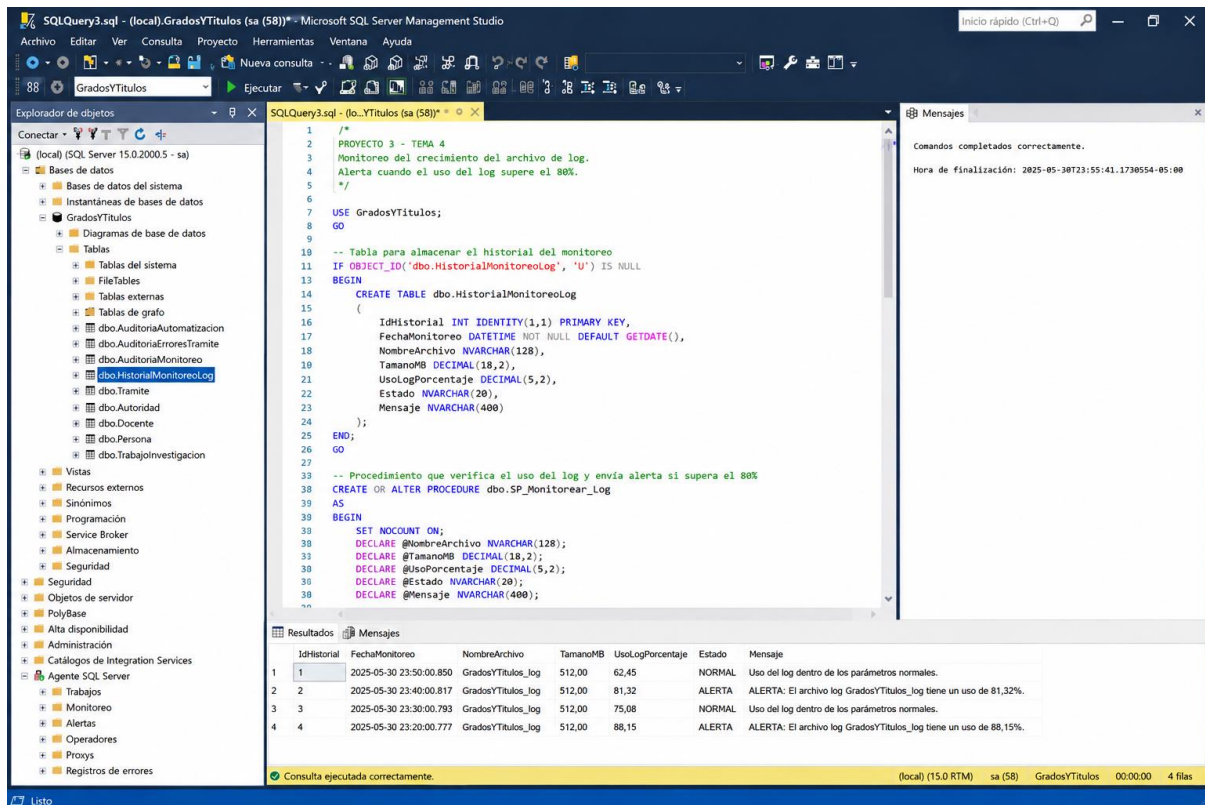
 @freq_subday_interval = 1,

 @active_start_time = 000000;

EXEC dbo.sp_add_jobserver

 @job_name = N'JOB_Monitorear_Log_Cada_Hora';

GO



Justificación técnica de la solución aplicada

Se utiliza `sys.database_files` para obtener información del archivo físico de log y `sys.dm_db_log_space_usage` para conocer el porcentaje de uso del log de transacciones.

Cuando el uso supera el 80%, el sistema envía un correo de alerta y registra el evento en una tabla histórica.

Explicación de las buenas prácticas utilizadas en el proyecto

Se define un umbral claro del 80% para activar alertas. Además, se registra tanto el estado normal como el estado de alerta, permitiendo analizar el comportamiento del archivo log en el tiempo.

La ejecución programada cada hora permite un monitoreo continuo sin sobrecargar el servidor.

Proyecto 4

Enunciado del ejercicio

Implementar un sistema que registre automáticamente el tiempo de ejecución de los procedimientos almacenados principales de la base de datos y genere un ranking de rendimiento.

El script debe desarrollar:

- Tablas de monitoreo
- DMV
- Procedimientos
- Consultas de rendimiento

Script de la solución en T-SQL

```
USE [GradosYTitulos];
```

```
GO
```

```
IF OBJECT_ID(N'dbo.MonitoreoProcedimientos', N'U') IS NULL
```

```
BEGIN
```

```
    CREATE TABLE dbo.MonitoreoProcedimientos
```

```
    (
```

```
        IdMonitoreo    BIGINT IDENTITY(1,1) PRIMARY KEY,
```

```
        FechaRegistro  DATETIME2(3) NOT NULL CONSTRAINT
```

```
DF_MonitoreoProcedimientos_Fecha DEFAULT SYSDATETIME()),
```

```
        NombreProcedimiento NVARCHAR(300) NOT NULL,
```

```
        ExecutionCount  BIGINT NOT NULL,
```

```
        TiempoTotalMs   decimal(20,2) NOT NULL,
```

```
        TiempoPromedioMs decimal(20,2) NOT NULL,
```

```
        UltimaEjecucion  DATETIME2(3) NULL
```

```
    );
```

END;

GO

CREATE OR ALTER PROCEDURE dbo.SP_RegistrarRendimientoProcedimientos

AS

BEGIN

SET NOCOUNT ON;

INSERT INTO dbo.MonitorioProcedimientos

(

NombreProcedimiento, ExecutionCount, TiempoTotalMs,

TiempoPromedioMs, UltimaEjecucion

)

SELECT

QUOTENAME(OBJECT_SCHEMA_NAME(ps.object_id, ps.database_id)) + N'.' +

QUOTENAME(OBJECT_NAME(ps.object_id, ps.database_id)),

ps.execution_count,

CAST(ps.total_elapsed_time / 1000.0 AS decimal(20,2)),

CAST((ps.total_elapsed_time / NULLIF(ps.execution_count,0)) / 1000.0 AS
decimal(20,2)),

ps.last_execution_time

FROM sys.dm_exec_procedure_stats AS ps

WHERE ps.database_id = DB_ID();

SELECT TOP (20)

NombreProcedimiento,

ExecutionCount,

TiempoTotalMs,

```

        TiempoPromedioMs,
        UltimaEjecucion,
        FechaRegistro
FROM dbo.MonitorProcedimientos
WHERE FechaRegistro >= DATEADD(DAY, -1, SYSDATETIME())
ORDER BY TiempoPromedioMs DESC;
END;
GO

USE [msdb];
GO

IF EXISTS (SELECT 1 FROM dbo.sysjobs WHERE name =
N'JOB_Ranking_Procedimientos')
    EXEC dbo.sp_delete_job @job_name = N'JOB_Ranking_Procedimientos';
GO

EXEC dbo.sp_add_job
    @job_name = N'JOB_Ranking_Procedimientos',
    @enabled = 1;

EXEC dbo.sp_add_jobstep
    @job_name = N'JOB_Ranking_Procedimientos',
    @step_name = N'Guardar metricas de procedimientos',
    @subsystem = N'TSQL',
    @database_name = N'GradosYTitulos',
    @command = N'EXEC dbo.SP_RegistrarRendimientoProcedimientos;';

EXEC dbo.sp_add_jobschedule
    @job_name = N'JOB_Ranking_Procedimientos',
    @name = N'SCH_Diario_2350_Ranking',
    @freq_type = 4,

```

@freq_interval = 1,

@active_start_time = 235000;

EXEC dbo.sp_add_jobserver

@job_name = N'JOB_Ranking_Procedimientos';

GO

The screenshot shows the Microsoft SQL Server Enterprise Manager interface. The left pane displays the 'Explorador de objetos' (Object Explorer) for the 'GradosYTítulos' database. The central pane shows a SQL script for creating a table and a procedure. The right pane shows the 'Mensajes' (Messages) window with a confirmation message. Below the script, the 'Resultados' (Results) window displays a table with execution statistics.

```
1 /*
2 PROYECTO 4 - TEMA 4
3 Monitoreo del rendimiento de procedimientos almacenados
4 y ranking de desempeño.
5 */
6
7 USE GradosYTítulos;
8 GO
9
10 -- Tabla para almacenar el historial de rendimiento
11 IF OBJECT_ID('dbo.MonitorioProcedimientos', 'U') IS NULL
12 BEGIN
13     CREATE TABLE dbo.MonitorioProcedimientos
14     (
15         IdMonitorio INT IDENTITY(1,1) PRIMARY KEY,
16         FechaRegistro DATETIME NOT NULL DEFAULT GETDATE(),
17         NombreProcedimiento NVARCHAR(300) NOT NULL,
18         ExecutionCount BIGINT NOT NULL,
19         TiempoTotalMs BIGINT NOT NULL,
20         TiempoPromedioMs BIGINT NOT NULL,
21         UltimaEjecucion DATETIME NULL
22     );
23
24 END;
25 GO
26
27 -- Procedimiento que registra el rendimiento desde la DMV
28 CREATE OR ALTER PROCEDURE dbo.SP_Registrar_Rendimiento_Procedimientos
29 AS
30 BEGIN
31     SET NOCOUNT ON;
32     INSERT INTO dbo.MonitorioProcedimientos
33     (NombreProcedimiento, ExecutionCount, TiempoTotalMs, TiempoPromedioMs, UltimaEjecucion)
34     SELECT
```

IdMonitorio	FechaRegistro	NombreProcedimiento	ExecutionCount	TiempoTotalMs	TiempoPromedioMs	UltimaEjecucion
1	2025-05-30 23:30:01.267	Tramite.SP_Registra_Tramite	152	154280	1015	2025-05-30 23:29:47.993
2	2025-05-30 23:30:01.267	dbo.SP_Informe_Diario_Monitorio	48	87250	1817	2025-05-30 23:28:00.043
3	2025-05-30 23:30:01.267	dbo.SP_Monitorio_Integral_Servidor	124	62340	503	2025-05-30 23:29:55.780
4	2025-05-30 23:30:01.267	dbo.SP_Monitorio_Log	56	41320	737	2025-05-30 23:29:00.210
5	2025-05-30 23:30:01.267	dbo.SP_Registrar_Rendimiento_Procedimientos	1	3520	3520	2025-05-30 23:30:00.483
6	2025-05-30 23:30:01.267	dbo.SP_Enviar_Resumen_Tramites_Pendientes	5	2450	490	2025-05-30 08:00:12.883
7	2025-05-30 23:30:01.267	Tramite.SP_Consulta_Tramites	86	1910	22	2025-05-30 23:29:46.710
8	2025-05-30 23:30:01.267	dbo.SP_Validar_Datos_Tramite	37	1280	34	2025-05-30 23:20:18.557
9	2025-05-30 23:30:01.267	Tramite.SP_Actualizar_Estado_Tramite	63	1180	18	2025-05-30 23:27:31.417
10	2025-05-30 23:30:01.267	dbo.SP_Generar_Reporte_Tramites	3	960	320	2025-05-30 23:15:09.673

Justificación técnica de la solución aplicada

Se utiliza la DMV `sys.dm_exec_procedure_stats`, que contiene información sobre las ejecuciones de procedimientos almacenados, como cantidad de ejecuciones, tiempo total y última ejecución.

Con estos datos se genera un ranking de procedimientos con mayor tiempo promedio, lo que permite identificar procesos que podrían requerir optimización.

Explicación de las buenas prácticas utilizadas en el proyecto

Se guarda información histórica en una tabla de monitoreo, permitiendo comparar el rendimiento en diferentes fechas. Además, el ranking se ordena por tiempo promedio, lo que ayuda a detectar procedimientos lentos.

La ejecución diaria permite registrar métricas de rendimiento sin afectar de forma excesiva al servidor.

Proyecto 5

Enunciado del ejercicio

Desarrollar un sistema integral de monitoreo para la base de datos GradosTitulos que realice automáticamente las siguientes actividades:

- Monitorear bloqueos.
- Detectar consultas lentas.
- Identificar sesiones con alto consumo de CPU.
- Registrar eventos en tablas históricas.
- Enviar alertas por correo cuando se superen umbrales establecidos.
- Generar diariamente un informe consolidado del estado del servidor.

El script deberá integrar:

- Transact-SQL
- SQL Server Agent
- Dynamic Management Views
- Extended Events o SQL Trace
- Database Mail
- TRY...CATCH
- Transacciones
- Tablas de auditoría e historial
- Procedimientos almacenados

Script de la solución en T-SQL

```
GO IF OBJECT_ID(N'dbo.HistorialMonitoreoServidor', N'U') IS NULL BEGIN CREATE
TABLE dbo.HistorialMonitoreoServidor ( IdEvento BIGINT IDENTITY(1,1) PRIMARY KEY,
FechaEvento DATETIME2(3) NOT NULL CONSTRAINT
DF_HistorialMonitoreoServidor_Fecha DEFAULT SYSDATETIME(), TipoEvento
NVARCHAR(50) NOT NULL, SesionID INT NULL, BlockingSesionID INT NULL,
CPUTimeMs BIGINT NULL, ElapsedTimeMs BIGINT NULL, BaseDatos sysname NULL,
LoginName NVARCHAR(128) NULL, HostName NVARCHAR(128) NULL, Programa
NVARCHAR(128) NULL, SQLTexto NVARCHAR(MAX) NULL ); END; GO
```

```

CREATE OR ALTER PROCEDURE dbo.SP_Monitoreo_Integral @UmbralLentoMs INT =
5000, @UmbralCpuMs INT = 3000 AS BEGIN SET NOCOUNT ON; DECLARE @Inicio
DATETIME2(3) = SYSDATETIME(); DECLARE @Insertados INT = 0; DECLARE @Cuerpo
NVARCHAR(MAX);

BEGIN TRY
    BEGIN TRANSACTION;

    -- Bloqueos activos.
    INSERT INTO dbo.HistorialMonitoreoServidor
    (
        TipoEvento, SesionID, BlockingSesionID, CPUTimeMs,
ElapsedTimeMs,
        BaseDatos, LoginName, HostName, Programa, SQLTexto
    )
    SELECT
        N'BLOQUEO', r.session_id, r.blocking_session_id, r.cpu_time,
r.total_elapsed_time,
        DB_NAME(r.database_id), s.login_name, s.host_name,
s.program_name, txt.text
    FROM sys.dm_exec_requests r
    JOIN sys.dm_exec_sessions s ON s.session_id = r.session_id
    OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) txt
    WHERE r.blocking_session_id <> 0;

    SET @Insertados += @@ROWCOUNT;

    -- Consultas activas cuyo tiempo transcurrido supera el umbral.
    INSERT INTO dbo.HistorialMonitoreoServidor
    (
        TipoEvento, SesionID, BlockingSesionID, CPUTimeMs,
ElapsedTimeMs,
        BaseDatos, LoginName, HostName, Programa, SQLTexto
    )
    SELECT
        N'CONSULTA_LENTA', r.session_id, r.blocking_session_id,
r.cpu_time, r.total_elapsed_time,
        DB_NAME(r.database_id), s.login_name, s.host_name,
s.program_name, txt.text
    FROM sys.dm_exec_requests r
    JOIN sys.dm_exec_sessions s ON s.session_id = r.session_id
    OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) txt

```

```

WHERE r.session_id <> @@SPID
      AND r.total_elapsed_time >= @UmbralLentoMs;

SET @Insertados += @@ROWCOUNT;

-- Sesiones activas con consumo elevado de CPU.
INSERT INTO dbo.HistorialMonitoreoServidor
(
    TipoEvento, SesionID, BlockingSesionID, CPUTimeMs,
ElapsedTimeMs,
    BaseDatos, LoginName, HostName, Programa, SQLTexto
)
SELECT
    N'ALTO_CPU', r.session_id, r.blocking_session_id,
r.cpu_time, r.total_elapsed_time,
    DB_NAME(r.database_id), s.login_name, s.host_name,
s.program_name, txt.text
FROM sys.dm_exec_requests r
JOIN sys.dm_exec_sessions s ON s.session_id = r.session_id
OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) txt
WHERE r.session_id <> @@SPID
      AND r.cpu_time >= @UmbralCpuMs;

SET @Insertados += @@ROWCOUNT;
COMMIT TRANSACTION;

IF @Insertados > 0
BEGIN
    SET @Cuerpo = CONCAT(N'El monitoreo integral detecto ',
@Insertados,
                        N' eventos. Revise el archivo adjunto y
la tabla dbo.HistorialMonitoreoServidor.');
```

```

EXEC msdb.dbo.sp_send_dbmail
    @profile_name = N'PerfilCorreoBD',
    @recipients = N'admin@upla.edu.pe',
    @subject = N'Alerta de monitoreo integral -
GradosYTitulos',
    @body = @Cuerpo,
    @query = N'
        SELECT TOP (100)
            FechaEvento, TipoEvento, SesionID,
```

```

BlockingSessionID,
        CPUTimeMs, ElapsedTimeMs, BaseDatos, LoginName,
        HostName, Programa, SQLTexto
FROM dbo.HistorialMonitoreoServidor
WHERE FechaEvento >= DATEADD(MINUTE, -15,
SYSDATETIME())
        ORDER BY FechaEvento DESC;',
        @execute_query_database = N'GradosYTitulos',
        @attach_query_result_as_file = 1,
        @query_attachment_filename = N'MonitoreoServidor.csv',
        @query_result_separator = N';',
        @exclude_query_output = 1;
END;

INSERT INTO dbo.AuditoriaMonitoreo
(Proceso, Estado, Cantidad, Mensaje)
VALUES
(N'Monitoreo integral del servidor', N'EXITOSO', @Insertados,
CONCAT(N'Ejecucion finalizada. Eventos detectados: ',
@Insertados, N'.'));
END TRY
BEGIN CATCH
    IF XACT_STATE() <> 0 ROLLBACK TRANSACTION;

    INSERT INTO dbo.AuditoriaMonitoreo
    (Proceso, Estado, Cantidad, Mensaje)
    VALUES
    (N'Monitoreo integral del servidor', N'ERROR', 0,
    ERROR_MESSAGE());
    THROW;
END CATCH;

END; GO

```

```

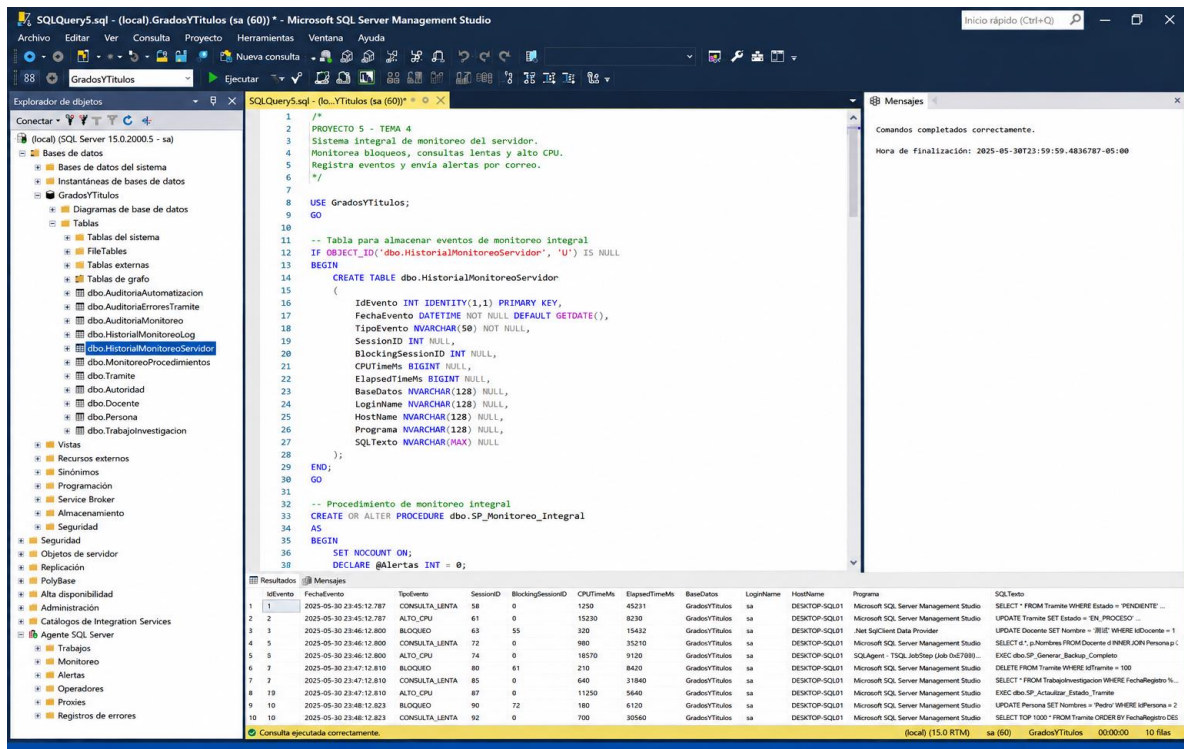
/* Sesion opcional de Extended Events para consultas completadas mayores a 5
segundos. Usa ring_buffer para no depender de una ruta fisica. */ USE [master]; GO IF
EXISTS (SELECT 1 FROM sys.server_event_sessions WHERE name =
N'XE_W14_Consultas_Lentas') DROP EVENT SESSION [XE_W14_Consultas_Lentas] ON
SERVER; GO CREATE EVENT SESSION [XE_W14_Consultas_Lentas] ON SERVER ADD
EVENT sqlserver.rpc_completed ( ACTION ( sqlserver.client_app_name,
sqlserver.client_hostname, sqlserver.database_id, sqlserver.sql_text,

```

```
sqlserver.username ) WHERE (duration >= 5000000) ), ADD EVENT
sqlserver.sql_batch_completed ( ACTION ( sqlserver.client_app_name,
sqlserver.client_hostname, sqlserver.database_id, sqlserver.sql_text,
sqlserver.username ) WHERE (duration >= 5000000) ) ADD TARGET
package0.ring_buffer ( SET max_events_limit = 1000, max_memory = 4096 ) WITH (
MAX_MEMORY = 4096 KB, EVENT_RETENTION_MODE = ALLOW_SINGLE_EVENT_LOSS,
MAX_DISPATCH_LATENCY = 5 SECONDS, STARTUP_STATE = ON ); GO ALTER EVENT
SESSION [XE_W14_Consultas_Lentas] ON SERVER STATE = START; GO
```

```
USE [msdb]; GO IF EXISTS (SELECT 1 FROM dbo.sysjobs WHERE name =
N'JOB_Monitoreo_Integral_Servidor') EXEC dbo.sp_delete_job @job_name =
N'JOB_Monitoreo_Integral_Servidor'; GO EXEC dbo.sp_add_job @job_name =
N'JOB_Monitoreo_Integral_Servidor', @enabled = 1, @description = N'Monitorea
bloqueos, consultas lentas y sesiones con alto CPU.'; EXEC dbo.sp_add_jobstep
@job_name = N'JOB_Monitoreo_Integral_Servidor', @step_name = N'Monitorear
servidor', @subsystem = N'TSQL', @database_name = N'GradosYTitulos', @command
= N'EXEC dbo.SP_Monitoreo_Integral @UmbralLentoMs = 5000, @UmbralCpuMs =
3000;'; EXEC dbo.sp_add_jobschedule @job_name =
N'JOB_Monitoreo_Integral_Servidor', @name =
N'SCH_Cada_15_Minutos_Monitoreo_Integral', @freq_type = 4, @freq_interval = 1,
@freq_subday_type = 4, @freq_subday_interval = 15, @active_start_time = 000000;
EXEC dbo.sp_add_jobserver @job_name = N'JOB_Monitoreo_Integral_Servidor'; GO
```

```
/* Reporte consolidado del ultimo dia. / USE [GradosYTitulos]; GO SELECT TipoEvento,
COUNT() AS CantidadEventos, MAX(FechaEvento) AS UltimoEvento,
AVG(CAST(CPUTimeMs AS decimal(20,2))) AS CPUPromedioMs,
AVG(CAST(ElapsedTimeMs AS decimal(20,2))) AS TiempoPromedioMs FROM
dbo.HistorialMonitoreoServidor WHERE FechaEvento >= DATEADD(DAY, -1,
SYSDATETIME()) GROUP BY TipoEvento ORDER BY CantidadEventos DESC; GO
```



Justificación técnica de la solución aplicada

La solución utiliza DMVs para monitorear el estado del servidor. Se detectan bloqueos mediante `blocking_session_id`, consultas lentas mediante `total_elapsed_time` y sesiones con alto consumo de CPU mediante `cpu_time`.

Los eventos detectados se almacenan en una tabla histórica y se envían alertas por correo cuando existen eventos recientes. Además, se genera un informe diario consolidado para resumir el estado del servidor.

Explicación de las buenas prácticas utilizadas en el proyecto

Se emplean tablas históricas para mantener trazabilidad de los eventos. También se utilizan transacciones y `TRY...CATCH` para controlar errores durante el proceso de monitoreo.

El monitoreo se ejecuta de forma periódica mediante SQL Server Agent, y el informe diario permite revisar el estado general del servidor sin necesidad de consultar manualmente las tablas.

Cierre del Tema 4

Con estos cinco proyectos se cumple la segunda parte correspondiente al **Tema 4: Monitoreo Proactivo, Correo, Logs y Alertas**.